LOGO INSTITUCIONAL — pegar aquí

UNIVERSIDAD TECNOLÓGICA DE CHIHUAHUA SUR

Ingeniería en Mecatrónica · 10.º cuatrimestre · Grupo 10IME1M

INFORME INTEGRADOR
AVANCE 2

AETHER

Diseño e implementación de una línea de manufactura automotriz con verificación por visión y gestión trazable de inventario mediante un robot móvil autónomo

Dato	Información
Título del proyecto	Diseño e implementación de una línea de manufactura automotriz con verificación por visión y gestión trazable de inventario mediante un robot móvil autónomo
Asignatura	nombre de la asignatura — por completar
Docente	nombre del docente — por completar
Carrera y grupo	Ingeniería en Mecatrónica · 10IME1M
Integrantes (6)	Mario Iván Méndez Álvarez Sadher Isaac Torres Galindo Pedro José Pallares Muñoz David Gerardo Garnica Chaparro Juan Emmanuel Martínez Aguirre Uriel Armando Rivero Rivero
Periodo escolar	periodo — por completar
Fecha de entrega	fecha de entrega — por completar

Índice general

Índice general....2
Índice de figuras....4
Índice de tablas....4
Introducción....6
Siglas y términos usados....6
Capítulo 1. Diagnóstico y propuesta....8
1.1 Resumen....8
1.2 Abstract....8
1.3 Antecedentes y contexto del proyecto....9
1.4 Descripción del proceso o problema....9
1.5 Diagnóstico de necesidades....9
1.6 Requerimientos del cliente/usuario y especificaciones....10
1.7 Planteamiento del problema....11
1.8 Objetivo general....12
1.9 Objetivos específicos....12
1.10 Justificación....13
1.11 Alcances y limitaciones....13
1.12 Normatividad y criterios de seguridad aplicables....13
Capítulo 2. Diseño y desarrollo....15
2.1 Metodología / estrategia de desarrollo....15
2.2 Diagrama de flujo general del funcionamiento....15
2.3 Arquitectura general del proyecto....18
2.4 Diseño mecánico: CAD, dimensiones, materiales y ensamble....19
2.5 Diseño eléctrico: alimentación, protecciones, distribución y diagrama eléctrico....22
2.6 Diseño electrónico: sensores, actuadores, controladores, interfaces y comunicación....25
2.7 Diseño y programación del sistema de control....26
2.8 Diagrama de conexiones / cableado....27
2.9 Programas y algoritmos principales....30
2.9.1 Juicio estable de la celda....30
2.9.2 Secuencia del pistón y de la banda....30
2.9.3 Envío de observaciones en lotes....30
2.9.4 Navegación y ciclo del carrito (diseño)....30
2.10 Lista de materiales, cantidades, marca/modelo, costo y fuente de adquisición....31
2.11 Distribución física / layout / boceto de maqueta....38
2.12 Procedimiento de construcción, integración y puesta en marcha....38
2.12.1 Celda de manufactura....39
2.12.2 Carrito (Aether Inventory)....40
2.12.3 Software: prueba local en la laptop....41
2.13 Pruebas preliminares, fallas encontradas y ajustes realizados....43
Capítulo 3. Administración y dirección....46
3.1 Plan de trabajo y cronograma / Gantt....46
3.2 Responsabilidades de los integrantes....48
3.3 Recursos humanos, materiales, herramientas y servicios....48
3.4 Presupuesto y control de costos....48
3.5 Gestión de adquisiciones y disponibilidad de recursos....49
3.6 Riesgos, contingencias y acciones preventivas y correctivas....49
3.7 Control y seguimiento: programado vs. realizado....50
Capítulo 4. Resultados y evaluación....51

4.1 Resultados de las pruebas....51
4.2 Pruebas en condiciones normales y, cuando aplique, máximas de operación....51
4.3 Cumplimiento de objetivos y metas....51
4.4 Impacto técnico, económico o productivo....52
4.5 Costo y tiempo real de realización....52
4.6 Evaluación de factibilidad y cumplimiento normativo....52
4.7 Producto final: fotografías, evidencia de funcionamiento y enlace al video....52
Conclusiones....54
Conclusión individual de cada integrante....54
Referencias....55
Anexos....56
Anexo A. Códigos comentados....56
Anexo B. Planos y dibujos técnicos....56
Anexo C. Diagramas adicionales....56
Anexo D. Fichas técnicas y manuales relevantes....56
Anexo E. Formatos de pruebas y listas de verificación....57
Anexo F. Códigos de programación....57

Índice de figuras

Figura 1. Diagrama de flujo general del sistema: entradas, proceso, control, actuadores y salidas....16
Figura 2. Diagrama de flujo de la celda: inspección, juicio y accionamiento del pistón y de la banda....17
Figura 3. Diagrama de flujo del ciclo del carrito: acople, carga de seis fixtures, viaje y descarga. 18
Figura 4. Arquitectura de equipos, puertos y protocolos del sistema Aether....19
Figura 5. Carrito armado en el modelo 3D (hoja de armado, revisión 3.8)....20
Figura 6. Vista explosionada: cada pieza separada de su posición final....20
Figura 7. Estructura transparente: la torre del elevador, la batería y la electrónica por dentro...21
Figura 8. Vista lateral del carrito con cotas principales....21
Figura 9. Esquema eléctrico de potencia y protecciones del carrito....23
Figura 10. Esquema eléctrico de la celda: Raspberry Pi 4, relevadores, LOGO!, infrarrojo y electroválvula....24
Figura 11. Modelo 3D de la electrónica con sus cables (vista «Solo electrónica»)....26
Figura 12. Diagrama de tiempos de la celda para una pieza aprobada y una rechazada....27
Figura 13. Conexiones del ESP32 (1 de 2): motores a pasos, servo y LEDs....28
Figura 14. Conexiones del ESP32 (2 de 2): RS485, finales de carrera, QTR-8A, I²C y lectura de batería....29
Figura 15. Distribución física de la celda, el carrito y la mesa de descarga (vista superior)....38
Figura 16. Vista general de la celda armada....39
Figura 17. Tablero con el LOGO!, la fuente de 24 V y los relevadores....40
Figura 18. Pistón de scrap y su electroválvula....40
Figura 19. Perfiles de 3030 cortados y marco armado....40
Figura 20. Bandeja de electrónica armada en la mesa....40
Figura 21. Portada del panel web (servidor local)....41
Figura 22. Pantalla de la celda (celda.html) en prueba local: estado «Operando», juicio, producción, últimas piezas y alarmas; GPIO simulado....42
Figura 23. Lumina, el asistente del robot, en reposo (robot.html)....42
Figura 24. Inventario declarado (inventory.html)....43
Figura 25. Producto final funcionando: celda y carrito....53

Índice de tablas

Tabla 1. Siglas y términos....6
Tabla 2. Descripción del proceso....9
Tabla 3. Diagnóstico del problema por categoría....10
Tabla 4. Alternativas de solución evaluadas y descartadas....10
Tabla 5. Requerimientos y especificaciones, con su método de verificación....11
Tabla 6. Normas y criterios de seguridad....14
Tabla 7. Calendario de fases del curso....15
Tabla 8. Equipos del sistema y su función....19
Tabla 9. Parámetros mecánicos del carrito (hoja de armado rev. 3.8)....22
Tabla 10. Sensores, actuadores y controladores....25
Tabla 11. Parámetros del ciclo de la celda (valores por defecto del código)....26
Tabla 12. Lista de materiales del carrito y la estación (69 componentes de la hoja de armado)31
Tabla 13. Materiales adicionales de la hoja de costos que no figuran como componente de la hoja de armado....34

Tabla 14. Tornillería y consumibles (hoja de costos).....35
Tabla 15. Componentes de la celda de manufactura (por confirmar con el equipo de la celda) 36
Tabla 16. Estado de cada parte.....38
Tabla 17. Pruebas preliminares.....43
Tabla 18. Fallas encontradas y ajustes realizados.....44
Tabla 19. Programa del curso (Gantt programado).....46
Tabla 20. Actividades del proyecto (Gantt realizado).....46
Tabla 21. Responsabilidades por área.....48
Tabla 22. Recursos del proyecto.....48
Tabla 23. Presupuesto por subsistema y costo real.....48
Tabla 24. Adquisiciones.....49
Tabla 25. Riesgos principales.....49
Tabla 26. Control de avance contra el calendario.....50
Tabla 27. Resultados de las pruebas.....51
Tabla 28. Pruebas de operación.....51
Tabla 29. Cumplimiento de objetivos.....51
Tabla 30. Costo y tiempo: planeado y real.....52
Tabla 31. Conclusiones individuales (cinco renglones por integrante).....54
Tabla 32. Fichas técnicas relevantes.....56
Tabla 33. Formato de registro de pruebas.....57
Tabla 34. Lista de verificación antes de energizar el carrito.....57
Tabla 35. Mapa del código.....57

Los tres índices son automáticos de Word (se arman con los estilos de título y con los títulos de figura y de tabla). Para actualizarlos: clic en el índice y tecla F9.

Introducción

Aether es el proyecto integrador de un equipo de seis alumnos de Ingeniería en Mecatrónica de la Universidad Tecnológica de Chihuahua Sur. Es una línea de manufactura pequeña que une tres cosas que normalmente se resuelven por separado: la **inspección de piezas con visión artificial**, la **expulsión automática de las piezas malas** y el **transporte de las piezas buenas con un robot móvil**, todo con un registro trazable en un servidor.

El proyecto cumple las tres condiciones del curso: un controlador lógico programable (PLC, aquí el LOGO! de Siemens) maneja la banda de la celda; microcontroladores y computadoras de borde (ESP32, Raspberry Pi 4 y Jetson Orin Nano Super) hacen el control y la visión; y la línea es de manufactura flexible porque las piezas se identifican por código QR y se clasifican por visión, sin programar una trayectoria fija por pieza.

Este documento es el **Avance 2** del informe integrador y sigue la estructura que pidió el docente: elementos iniciales, cuatro capítulos (diagnóstico y propuesta, diseño y desarrollo, administración y dirección, resultados y evaluación), conclusiones, referencias y anexos. Cada afirmación lleva su estado: **VERIFICADO** (se vio en una salida real al preparar este documento), **DISEÑO** (está definido en la hoja de armado o en el código, pero no se ha medido en el equipo físico) o **PENDIENTE**. Los datos que el equipo debe completar están marcados en amarillo.

Siglas y términos usados

Las siglas se explican en el texto la primera vez que aparecen y además se reúnen aquí.

Tabla 1. Siglas y términos

Sigla o término	Significado
ADR	Registro de decisión de arquitectura: documento corto donde el equipo anota qué decidió y por qué.
API / REST	Forma en que un programa pide datos a otro por la red; REST es el estilo que usa el servidor del proyecto.
ArUco	Marcador cuadrado en blanco y negro que una cámara reconoce para saber su posición.
AWG	Calibre estadounidense de cable: a menor número, más grueso.
BOM	Lista de materiales (Bill of Materials).
CAD	Diseño asistido por computadora.
ECU	Unidad de control electrónico: la pieza automotriz que se inspecciona y se transporta.
ESP32	Microcontrolador que mueve los motores y lee los sensores del carrito.
Fixture	Soporte con quilla donde viaja cada ECU; es lo que toca la banda y lo que carga el carrito.
GPIO	Pin de entrada o salida de uso general de una tarjeta.
HMI	Interfaz hombre-máquina: la pantalla con la que el operador ve y controla la celda.
I²C	Bus de dos cables para conectar sensores a un microcontrolador.

Sigla o término	Significado
IMU	Unidad de medición inercial: mide giros y aceleraciones (aquí, el BNO055).
IR	Infrarrojo: sensor que detecta si hay una pieza frente a él.
ISA-95	Norma que ordena los niveles de una fábrica, desde el sensor hasta la gestión.
JSON	Formato de texto para intercambiar datos entre programas.
LOGO!	Controlador lógico programable (PLC) pequeño de Siemens; maneja la banda de la celda.
MAX485 / RS485	Circuito y estándar de comunicación serial de dos hilos usado para hablar con los motores-rueda.
mAP50	Medida de qué tan bien detecta objetos un modelo de visión (1.0 es perfecto).
MDF	Tablero de fibra de madera de densidad media.
Modbus TCP	Protocolo industrial por red para leer y escribir registros en un controlador.
NEMA 17	Tamaño estándar de motor a pasos (carcasa de 42 mm).
NC / NO / COM	Contacto normalmente cerrado / normalmente abierto / común de un relevador o botón.
ONNX	Formato abierto para guardar un modelo de inteligencia artificial y correrlo en distintos equipos.
PASS / FAIL	Resultado de la inspección: pieza aprobada / pieza rechazada.
PETG	Plástico para impresión 3D, resistente y poco quebradizo.
PLC	Controlador lógico programable. En la celda es el LOGO!.
POST	Petición HTTP que envía datos a un servidor.
PWM	Modulación por ancho de pulso: forma de controlar el ángulo de un servo.
QR	Código cuadrado que se lee con cámara; cada fixture lleva uno.
Scrap	Pieza rechazada que se expulsa de la línea.
SQLite	Base de datos en un solo archivo, usada por el servidor.
STEP / DIR	Dos señales con las que un driver mueve un motor a pasos: un pulso por paso y el sentido de giro.
TR8	Husillo trapezoidal de 8 mm: convierte el giro del motor en subida y bajada.
VL53L0X	Sensor láser de distancia de corto alcance.
YOLO	Modelo de inteligencia artificial que detecta objetos en una imagen de una sola pasada.

Capítulo 1. Diagnóstico y propuesta

1.1 Resumen

Aether es un proyecto integrador que desarrolla una línea de manufactura automotriz: una banda transportadora con fixtures posiciona cada pieza, un sistema de visión confirma si quedó correctamente colocada, un pistón expulsa automáticamente la pieza si es scrap y un robot móvil autónomo, Aether Inventory (el carrito), recolecta y transporta las piezas terminadas mientras mantiene un registro trazable del inventario. La necesidad surge de la dependencia de operadores humanos en líneas pequeñas para trasladar piezas, verificar su ensamble y contar el inventario, lo que produce tiempos muertos, inconsistencia de calidad y errores acumulados.

La metodología siguió un orden de decisiones de arquitectura cerradas antes de programar (percepción híbrida YOLO más código de barras o QR, localización por ArUco, esquema de datos crudo frente a declarado), seguido de desarrollo por módulos probados por separado antes de integrarse. Al 24 de octubre de 2026, el software de la celda (visión, juicio PASS/FAIL y accionamiento del pistón y de la banda), el motor de inventario, el servidor y el panel corren y están verificados con pruebas automatizadas; el pistón ya funciona con el LOGO! de la celda según el equipo. El robot móvil está diseñado y documentado en 3D, pero **aún no está construido ni probado** y su firmware no existe. La conclusión principal es que la capa de datos y software es funcional y auditable, mientras que la construcción del robot define la ruta crítica restante.

1.2 Abstract

Aether is an integrator project that develops an automotive manufacturing line: a conveyor with positioning fixtures places each part, a vision system confirms whether it was placed correctly, a piston automatically ejects the part if it is scrap, and an autonomous mobile robot, Aether Inventory, collects and transports finished parts while keeping a traceable inventory record. The need arises from small lines depending on human operators to move parts, verify assembly and count inventory, which produces downtime, inconsistent quality checks and accumulated counting errors.

The methodology closed architecture decisions before implementation (hybrid YOLO plus barcode or QR perception, ArUco-based localization, a raw-versus-declared data schema), followed by module-by-module development tested independently before integration. As of October 24, 2026, the cell software (vision, PASS/FAIL judgment, piston and belt control), the inventory engine, the server and the dashboard run and are verified with automated tests, and the piston already works with the cell PLC (a Siemens LOGO!). The mobile robot is designed and documented in 3D but has not been built or tested yet. The main conclusion is that the data and software layer is functional and auditable, while building the robot defines the remaining critical path.

1.3 Antecedentes y contexto del proyecto

La necesidad se presenta en un contexto académico: el proyecto se desarrolla como integrador de la carrera de Ingeniería en Mecatrónica de la Universidad Tecnológica de Chihuahua Sur, simulando una línea de manufactura automotriz real. Aether se desarrolla en un contexto de manufactura con presencia significativa en Chihuahua, con demanda constante de líneas que combinen manipulación o transporte automatizado de piezas, verificación de calidad por visión y control de inventario.

Actualmente, en líneas de manufactura pequeñas y en entornos académicos equivalentes, la coordinación entre estaciones de trabajo depende de operadores humanos que trasladan piezas entre procesos, verifican visualmente el resultado de cada operación y llevan el conteo de inventario de forma manual. Esta forma de trabajo actual es el punto de partida que el proyecto busca mejorar.

El proyecto nació con dos brazos robóticos industriales. El 16 de septiembre de 2026 el equipo cambió el alcance (ADR 0010: registro de decisión de arquitectura) y sustituyó los brazos por una banda transportadora con fixtures y un pistón de expulsión, porque los brazos no cabían en el tiempo y el equipo disponibles. La visión, el inventario, el servidor y el robot móvil se mantuvieron.

1.4 Descripción del proceso o problema

Tabla 2. Descripción del proceso

Elemento	Descripción
Proceso actual	Un operador traslada la pieza entre estaciones, inspecciona visualmente si el resultado de cada operación es correcto y actualiza el conteo de inventario a mano.
Entrada	Piezas automovilísticas (unidades de control electrónico, ECU) sobre su fixture.
Subsistemas de la propuesta	Banda transportadora con fixtures (posicionamiento), estación de visión (verificación binaria PASS/FAIL), pistón de expulsión (manejo de scrap) y robot móvil (recolección, transporte y conteo).
Salida	Piezas terminadas, clasificadas y contadas en un inventario trazable.
Condiciones de operación	Entorno de laboratorio controlado, iluminación de interior, operación supervisada.
Restricción principal	Ventana de desarrollo de aproximadamente 240 horas en 4 meses, con un solo integrante a cargo de todo el software.

1.5 Diagnóstico de necesidades

La dependencia del operador humano produce **cuatro problemas recurrentes**, diagnosticados por categoría en la tabla siguiente: personal, método, detección y entorno.

Tabla 3. Diagnóstico del problema por categoría

N.º	Categoría	Hallazgo / riesgo	Cómo lo atiende Aether
1	Personal	La dependencia de operadores humanos para trasladar piezas entre estaciones y para contar inventario introduce variabilidad: el criterio y la velocidad cambian de una persona a otra y a lo largo del turno.	El robot traslada los fixtures y cada observación queda registrada en el servidor.
2	Método	Sin un protocolo de enlace entre estaciones, cada operación queda aislada y el operador humano se convierte en el mecanismo de coordinación por defecto, lo que genera tiempos muertos entre procesos.	Un flujo único celda → robot → mesa, con eventos con identificador (event_id) en el servidor.
3	Detección	Con reflejo de luz sobre un marcador ArUco, el detector puede reportar un identificador incorrecto pero válido dentro del diccionario, sin señal de error visible. Hallazgo verificado durante las pruebas del módulo de percepción.	El juicio PASS/FAIL exige 5 resultados iguales seguidos; el ArUco solo confirma posición.
4	Entorno	La variabilidad de iluminación y la falta de puntos fijos de referencia en un entorno de manufactura no controlado incrementan el riesgo de error en la detección de objetos y en la verificación de posición.	Marcadores ArUco, embudos de acople y sensores de distancia como respaldo; calibración en la planta (pendiente).

Antes de fijar la arquitectura se evaluaron y descartaron formalmente varias alternativas, resumidas en la tabla siguiente.

Tabla 4. Alternativas de solución evaluadas y descartadas

Alternativa descartada	Decisión tomada	Razón
Aprendizaje por demostración (LfD) para manipulación de piezas	Automatización fija de banda, fixtures y pistón	El alcance académico no requiere generalizar tareas nuevas; un diseño fijo da un comportamiento predecible y verificable.
ArUco como sistema de navegación completo	ArUco como capa de verificación de posición; navegación por waypoints y encoders	Un sistema de navegación basado solo en marcadores es frágil ante oclusiones y errores de lectura.
SLAM (mapeo y localización simultáneos) para la navegación del robot móvil	Diferido como característica futura	Excede el tiempo disponible del proyecto y no es indispensable para el ciclo completo en laboratorio.
Clasificación visual pura de producto, sin código	Enfoque híbrido YOLO más código de barras o QR	Requeriría un dataset de entrenamiento propio por tipo de pieza, esfuerzo no justificado frente al enfoque híbrido.
Orquestador central para las estaciones de línea	Protocolo de enlace punto a punto (I/O digital más MQTT)	Un orquestador central agrega un punto único de falla no justificado para una secuencia fija de estaciones.

1.6 Requerimientos del cliente/usuario y especificaciones

La tabla siguiente convierte la necesidad diagnosticada en requisitos comprobables. El «cliente» es la institución y el docente que evalúan el proyecto y el «usuario» es la persona que opera la línea. Los requisitos que ya cuentan con una prueba real se marcan como verificados; los que

dependen de hardware físico pendiente se marcan como pendientes de medición, siguiendo el principio de honestidad del proyecto de no reportar como cumplido lo que no se ha probado.

Tabla 5. Requerimientos y especificaciones, con su método de verificación

Requisito	Condición	Verificación
Detección combinada de objetos	YOLO detecta y clasifica el objeto en el fotograma	Verificado: test_combined.py sobre casos de prueba reales (Avance 1)
Identificación exacta cuando hay código	Lectura de código de barras o QR dentro de la caja detectada	Verificado: 712 Observations de una corrida real, con al menos una identificada por código (Avance 1)
Separación de datos crudo y declarado	Ninguna corrida borra automáticamente una entrada declarada	Verificado: pruebas automatizadas del backend (el retiro nunca borra la fila)
Trazabilidad histórica	Cada corrida agrega una fila nueva a la bitácora, sin sobrescribir	Verificado: prueba de orden de /inventory/history (Avance 1)
Operación de retiro protegida	Requiere contraseña válida; rechaza con 403 si es incorrecta	Verificado: pruebas de contraseña correcta e incorrecta (Avance 1)
Juicio PASS/FAIL estable	5 juicios iguales seguidos tomados cada 0.5 s (≈ 2.5 s por pieza)	Implementado y probado en código; prueba con la banda real PENDIENTE
Expulsión de piezas FAIL	Pistón activado 1 s tras una espera de 3 s; regreso en 2 s; falla registrada como «fallo_actuacion»	Implementado en código; el pistón ya funciona con el LOGO! (según el equipo); calibración en banda PENDIENTE
Operación de la celda sin la laptop	Juicio y pistón en la Raspberry Pi 4	Implementado en código; prueba PENDIENTE
Transporte de fixtures	6 fixtures por carga; entrega a 83 cm; descarga en mesa de 140 × 30 cm	DISEÑO; robot sin construir
Paro de emergencia del robot	Contacto normalmente cerrado que corta ruedas y motores a pasos; watchdog de 1 s	DISEÑO; prueba PENDIENTE
Precisión de la verificación por visión	Umbral de precisión mínimo, aún por definir	Pendiente: depende del documento de criterios de aceptación, en construcción
Capacidad de la línea (piezas por minuto)	Umbral mínimo, aún por definir	Pendiente: requiere banda transportadora y pistón físicos
Carga máxima del robot móvil	Estimado preliminar: máximo 40 kg totales, hasta 17 kg por llanta motriz	Pendiente: estimación preliminar; M0601 decidido, sin medir

1.7 Planteamiento del problema

Las líneas de manufactura pequeñas, y los entornos académicos que las simulan, dependen de operadores humanos para tres tareas que se repiten en cada ciclo de producción: trasladar las piezas entre estaciones, verificar visualmente si el resultado del proceso es correcto y llevar a mano el conteo del inventario. Cada una de esas tareas se hace con criterio y velocidad que cambian de una persona a otra y a lo largo del turno.

Esta dependencia produce tiempos muertos, porque cada estación espera a que alguien le traiga o le retire la pieza; inconsistencia en la verificación de calidad, porque no hay un criterio único y medible; y errores acumulados en los registros de inventario, que se agravan conforme aumenta el volumen de producción. Además, como no existe un enlace definido entre estaciones, el

operador se vuelve el mecanismo de coordinación por defecto, y el proceso no puede auditarse: no hay manera de saber qué pieza se rechazó, cuándo ni por qué.

Las soluciones comerciales que resuelven este problema por completo suelen superar el presupuesto de una pequeña o mediana empresa, y las soluciones parciales dejan huecos de coordinación que se terminan cubriendo de nuevo con trabajo manual. Por eso hace falta una solución integrada y accesible que inspeccione cada pieza con un criterio uniforme, retire automáticamente la pieza defectuosa, transporte las piezas aprobadas sin intervención humana y deje un registro trazable de todo el proceso.

El proyecto plantea resolver estos problemas con una banda transportadora con fixtures y un sistema de visión para el posicionamiento y la verificación de calidad, un pistón que expulsa la pieza si es scrap y un robot móvil autónomo para el transporte y el seguimiento del inventario, comunicados entre sí sin depender de intervención humana en el flujo normal de operación. El problema se delimita a un entorno de laboratorio controlado, con operación supervisada y piezas de una sola familia (unidades de control electrónico).

1.8 Objetivo general

Desarrollar una línea de manufactura automotriz funcional que integre una banda transportadora con fixtures de posicionamiento, un sistema de visión artificial de verificación, un pistón de expulsión de piezas defectuosas y una unidad móvil de inventario, capaz de ejecutar el ciclo completo de posicionamiento, verificación y transporte de piezas terminadas sin orquestación central, y de mantener un registro trazable del inventario resultante.

1.9 Objetivos específicos

- Analizar el proceso manual actual de traslado, verificación y conteo de piezas para identificar sus puntos de falla.
- Diseñar un sistema de percepción híbrido capaz de detectar y contar objetos mediante YOLO, e identificarlos de forma exacta mediante lectura de código de barras o QR cuando esté disponible.
- Seleccionar un sistema de localización basado en marcadores ArUco para verificación de posición, sin depender de este sistema para la navegación del robot.
- Controlar la banda transportadora con fixtures de posicionamiento y el pistón de expulsión de piezas defectuosas mediante el control de bajo nivel definido en la arquitectura del proyecto.
- Desarrollar un sistema de visión de línea que confirme de forma binaria si una pieza quedó correctamente colocada.
- Integrar un esquema de datos que distinga entre observaciones crudas de percepción e inventario declarado, preservando la trazabilidad histórica de cada corrida de inspección.

- Probar y validar cada módulo por separado antes de integrarlo mediante un protocolo de enlace punto a punto, sin un orquestador central.
- Evaluar el cumplimiento de los requerimientos de la sección 1.6 conforme el hardware físico esté disponible.

1.10 Justificación

La automatización de líneas de manufactura pequeñas suele encontrarse con una disyuntiva: las soluciones comerciales completas superan el presupuesto de una pequeña o mediana empresa, mientras que las soluciones parciales dejan huecos de coordinación que terminan resolviéndose con intervención manual. Aether explora una tercera vía, con alcance académico: construir cada módulo con tecnología accesible y bien documentada (Python, OpenCV, YOLO, FastAPI, MQTT) e integrar los módulos mediante un protocolo de enlace simple en vez de un orquestador central costoso.

El beneficio no está únicamente en la automatización física, que es la parte más visible de una línea de manufactura, sino en la capa de datos que la acompaña: un inventario que distingue entre lo observado y lo declarado, que nunca sobrescribe silenciosamente un resultado anterior con uno parcial y que mantiene una bitácora completa de cada corrida. Esta capa de datos hace que la línea sea auditable, y es una de las contribuciones específicas del proyecto más allá de la automatización convencional.

1.11 Alcances y limitaciones

El alcance obligatorio del proyecto académico es una línea de manufactura completa: la banda transportadora posiciona las piezas sobre sus fixtures, el sistema de visión confirma binariamente si la colocación es correcta, el pistón expulsa la pieza si es scrap y Aether Inventory recoge la pieza terminada, la transporta y lleva el control de todo el proceso. El desarrollo del software se ejecuta bajo una ventana de aproximadamente 240 horas en 4 meses, con un solo integrante a cargo de todo el software y la visión computacional.

Como limitación explícita, la primera versión del sistema no necesita resolver el problema general de navegación en cualquier almacén: el entorno inicial es controlado (laboratorio de la universidad), se prioriza una solución funcional y demostrable sobre una solución general, y las características avanzadas (SLAM, múltiples robots, operación en múltiples instalaciones) quedan fuera de este alcance y se documentan como trabajo futuro. Al 24 de octubre de 2026 el robot no está construido ni su firmware escrito, las distancias entre estaciones y todos los tiempos deben calibrarse en la planta, el enlace Modbus TCP entre la Raspberry Pi y el LOGO! es un diseño que no está en el código y no se ha hecho una evaluación formal de riesgos.

1.12 Normatividad y criterios de seguridad aplicables

El proyecto define un principio de seguridad desde su arquitectura de comunicaciones: el comando de parada de seguridad no depende de la conectividad de red, sino de un watchdog

(temporizador de vigilancia) implementado en el microcontrolador de bajo nivel, de forma que una falla de comunicación nunca deje al sistema sin capacidad de detenerse. Además se toman como referencia de diseño las normas de la tabla siguiente.

Tabla 6. Normas y criterios de seguridad

Norma o criterio	Qué pide, en palabras sencillas	Cómo se aplica en Aether	Estado
ISO 12100	Evaluar los riesgos de una máquina y reducirlos desde el diseño	Paro de emergencia, protecciones y separación de ramas de energía	Aplicada en el diseño; evaluación formal PENDIENTE
ISO 13849-1	Que las funciones de seguridad del control sean confiables	El paro corta la energía por hardware (relevador), sin depender del programa	Diseño
NOM-004-STPS	Proteger al personal de la maquinaria	Paro de emergencia accesible y piezas móviles identificadas	Diseño; verificación PENDIENTE
NOM-001-SEDE	Instalaciones eléctricas seguras	Fusibles por rama, calibres de cable, 24 V separados de los 5 V	Diseño
ISO 3691-4	Seguridad de vehículos industriales sin conductor	Frenado por sensores a menos de 40 cm y velocidad limitada a 0.35 m/s	Diseño; prueba PENDIENTE
ISA-95 / ISA-101 / ISA-18.2	Niveles de una fábrica, pantallas de operador y manejo de alarmas	Capas del sistema y alarmas de prioridad en la HMI	Aplicada parcialmente

Capítulo 2. Diseño y desarrollo

2.1 Metodología / estrategia de desarrollo

El equipo trabaja por módulos: cada módulo (visión, celda, servidor, carrito) se prueba solo antes de integrarse con el siguiente. Las decisiones importantes se registran en ADR y los cambios de alcance siguen un proceso de cambios documentado. Cada afirmación del informe se etiqueta como VERIFICADO, DISEÑO o PENDIENTE para no declarar como hecho lo que no se ha visto funcionar. El calendario del curso fija las fases de la tabla siguiente.

Tabla 7. Calendario de fases del curso

Fecha	Fase o entrega
11 de septiembre	Integración de equipos e idea del proyecto (cumplir con PLC, microcontrolador y manufactura flexible)
25 de septiembre	Definición y diseño conceptual
9 de octubre	Diseño técnico y planeación
23 de octubre	Construcción e integración
6 de noviembre	Pruebas y validación
20 de noviembre	Entrega preliminar: proyecto funcional y documento completo
26 de noviembre	Pre-integradora (presentación en PowerPoint explicada a profesores)
27 de noviembre al 2 de diciembre	Correcciones finales
3 de diciembre	Presentación final de Integradora III (flyer y exposición de máximo 5 minutos)

2.2 Diagrama de flujo general del funcionamiento

La Figura 1 resume el sistema completo en cinco columnas: **entradas** (lo que se lee), **proceso** (lo que se hace con ello), **control** (quién decide), **actuadores** (lo que se mueve) y **salidas** (lo que se entrega). Cada franja horizontal es una parte del sistema: la celda, el carrito y el software. Los textos se tomaron del código de la celda y de la hoja de armado del carrito.

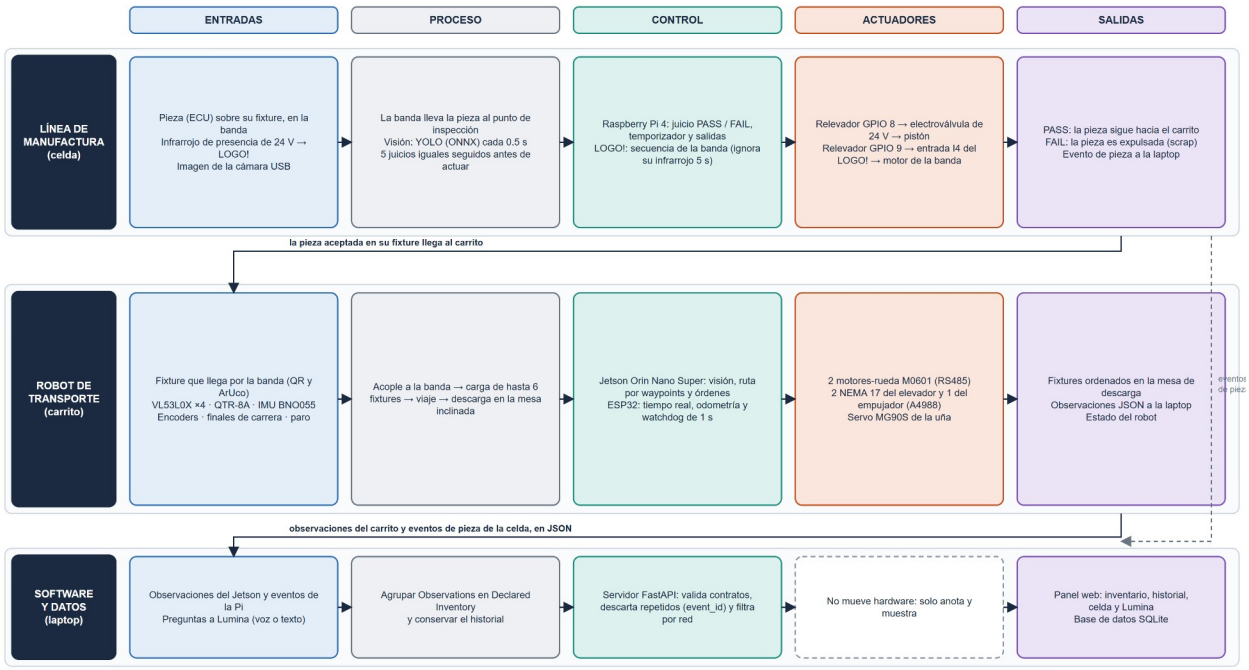


Figura 1. Diagrama de flujo general del sistema: entradas, proceso, control, actuadores y salidas.

La Figura 2 sigue paso a paso el programa de la celda (archivos cell_line.py y celda_agent.py). Una pieza solo se acepta o se rechaza después de **cinco juicios iguales seguidos**, tomados cada 0.5 segundos, para que un solo cuadro borroso no provoque una expulsión. Si el pistón no puede accionarse, el sistema no finge que lo hizo: registra la falla y levanta una alarma.

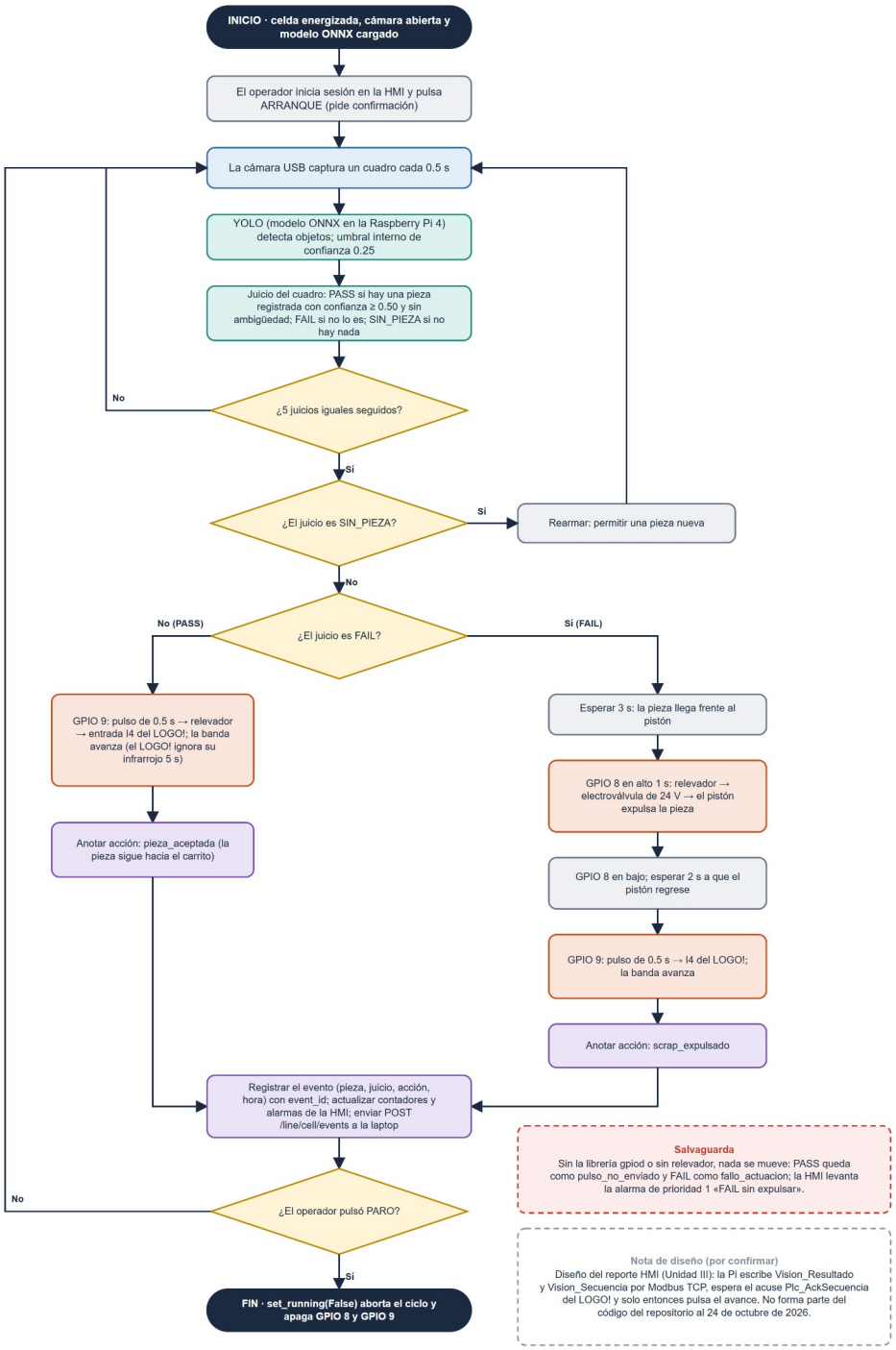


Figura 2. Diagrama de flujo de la celda: inspección, juicio y accionamiento del pistón y de la banda.

La Figura 3 describe el ciclo del carrito tal como lo define la hoja de armado (revisión 3.8). Es un **diseño**: el firmware del ESP32 todavía no existe, por lo que ningún paso se ha ejecutado en el carrito físico.

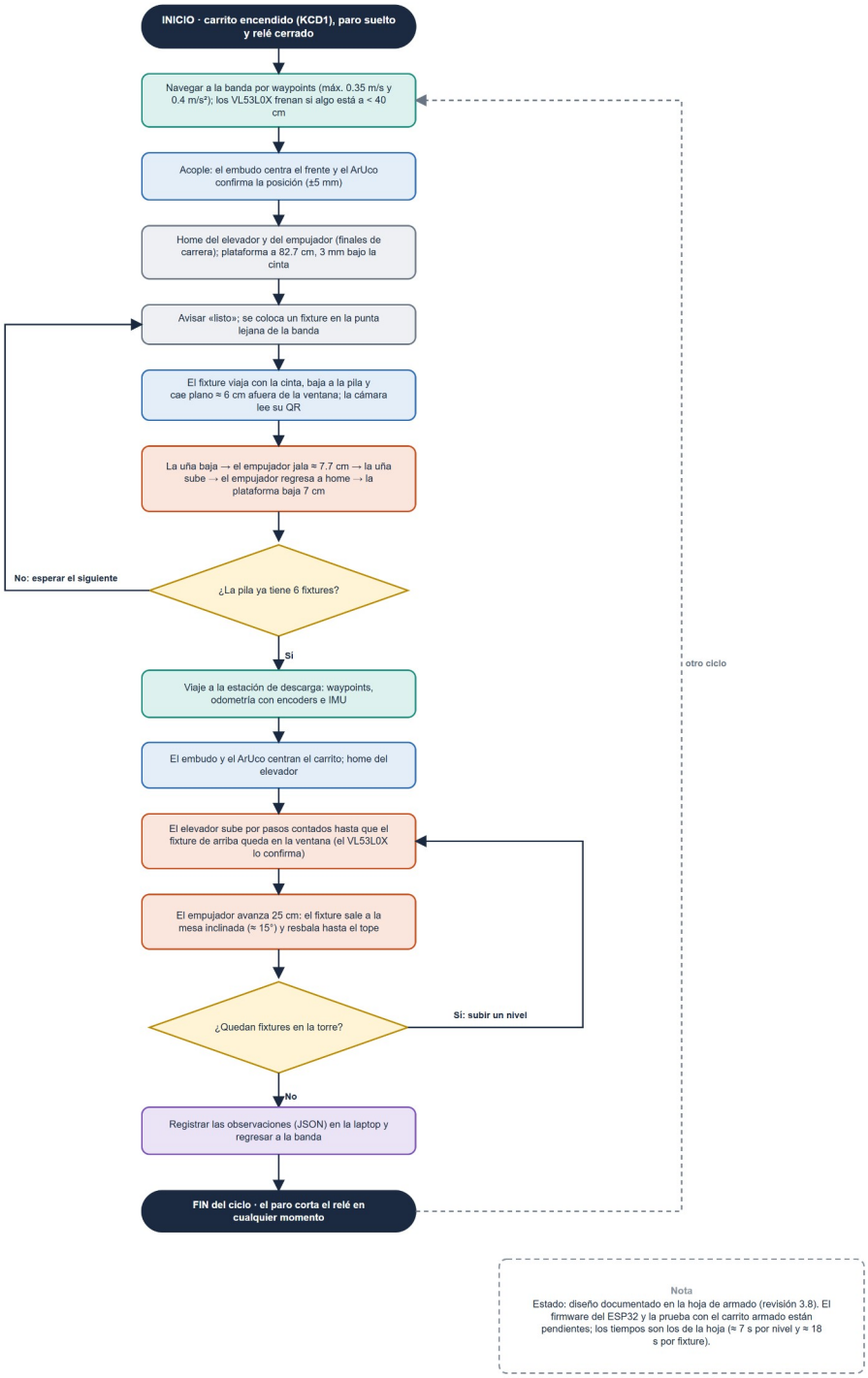


Figura 3. Diagrama de flujo del ciclo del carrito: acople, carga de seis fixtures, viaje y descarga.

2.3 Arquitectura general del proyecto

La Figura 4 muestra qué equipo ejecuta cada programa y por dónde se comunican. Todos los equipos están en una red local privada. Solo la síntesis de voz del asistente y el sitio público usan Internet; el sitio público es estático y no tiene acceso a la celda.

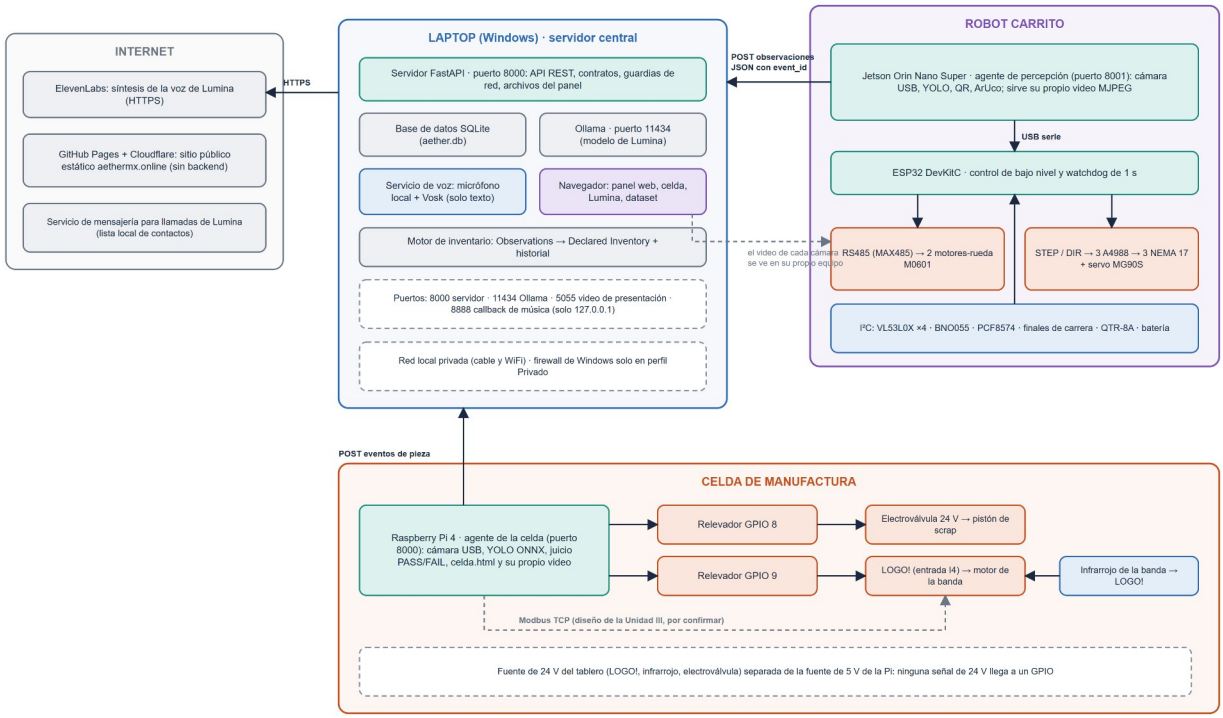


Figura 4. Arquitectura de equipos, puertos y protocolos del sistema Aether.

Tabla 8. Equipos del sistema y su función

Equipo	Programa que ejecuta	Función	Estado
Raspberry Pi 4 (celda)	Agente de la celda (modelo ONNX)	Cámara USB, juicio PASS/FAIL, relevadores GPIO 8 y 9, video propio	Código VERIFICADO en el repositorio; prueba con la banda completa PENDIENTE
LOGO! (celda)	Programa de la banda	Mueve el motor de la banda con la entrada I4 y el infrarrojo I3	Funciona con el pistón (según el equipo); programa por documentar
Jetson Orin Nano Super (carrito)	Agente de percepción (puerto 8001)	Cámara, YOLO, QR, ArUco; envía observaciones	PyTorch de NVIDIA funciona en el Jetson (bitácora del repositorio)
ESP32 (carrito)	Firmware (PlatformIO)	Motores a pasos, RS485, sensores, watchdog de 1 s	PENDIENTE: no existe el firmware
Laptop (servidor)	FastAPI, SQLite, Ollama, voz (Vosk)	Registro de observaciones, panel web y Lumina	Código VERIFICADO en el repositorio

2.4 Diseño mecánico: CAD, dimensiones, materiales y ensamble

El carrito se diseñó en 3D y su hoja de armado interactiva (revisión 3.8) reúne 69 componentes con su función, posición y comprobación de montaje. Las Figuras 5 a 7 son capturas de ese modelo y la Figura 8 muestra las cotas principales. La banda entrega los fixtures a 83 cm de altura por una ventana en el panel frontal; el elevador sube la pila por pasos hasta que el fixture superior queda frente a la ventana de descarga.



Figura 5. Carrito armado en el modelo 3D (hoja de armado, revisión 3.8).

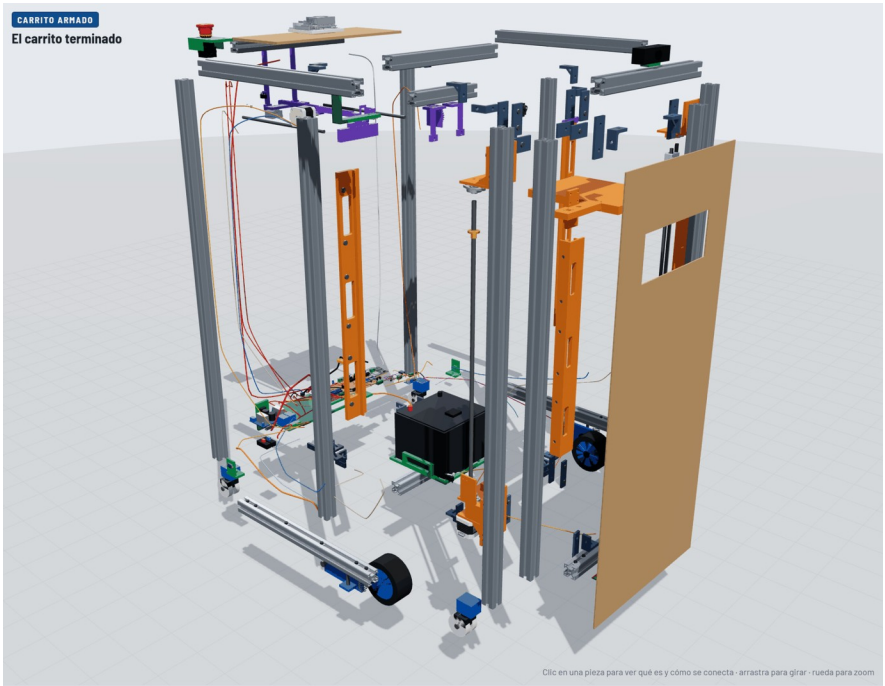


Figura 6. Vista explosionada: cada pieza separada de su posición final.

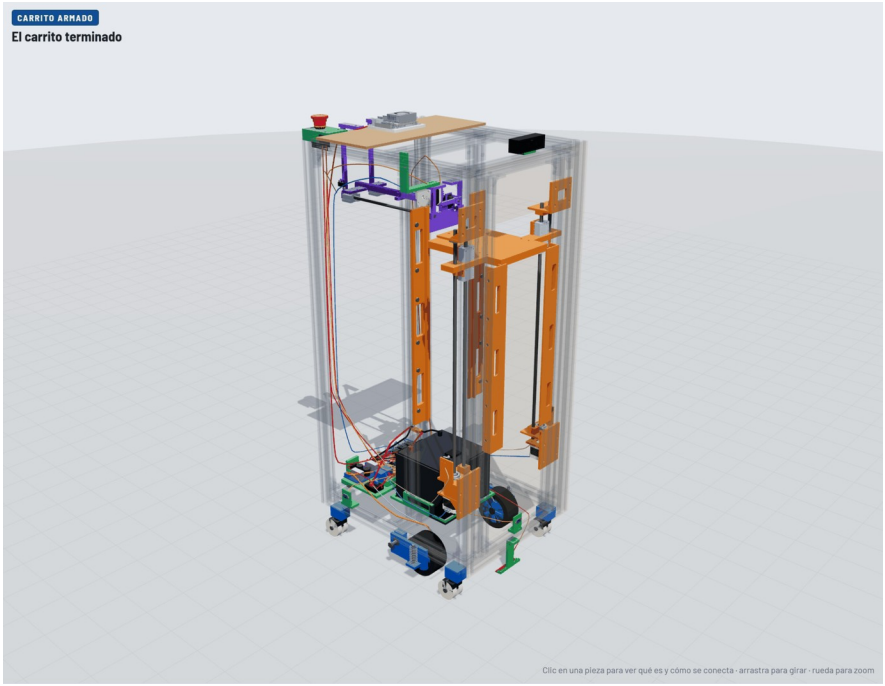


Figura 7. Estructura transparente: la torre del elevador, la batería y la electrónica por dentro.

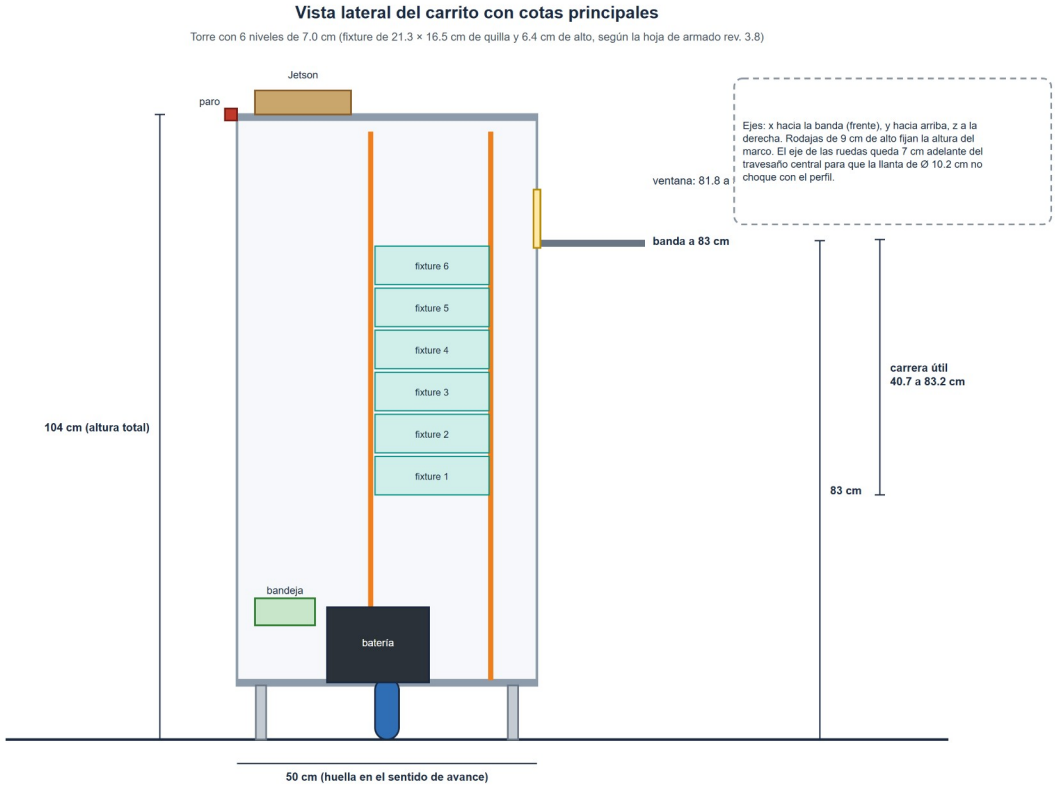


Figura 8. Vista lateral del carrito con cotas principales.

Tabla 9. Parámetros mecánicos del carrito (hoja de armado rev. 3.8)

Parámetro	Valor	Estado
Estructura y material	Perfil de aluminio 3030; marco de 0.50 × 0.40 m y 1.04 m de alto; escuadras y placas impresas en PETG; panel frontal de MDF de 3 mm	DISEÑO
Tracción	2 motores-rueda M0601 en suspensión con resorte (≈ 85 N por rueda) y 4 rodajas de nylon	DISEÑO; M0601 decidido y cotizado
Elevador	2 husillos TR8 de 60 cm con 2 NEMA 17; carrera útil de 40.7 a 83.2 cm	DISEÑO
Empujador	NEMA 17, piñón m1.5 de 20 dientes y cremallera; avance de 25 cm a 80 mm/s	DISEÑO
Uña del fixture	Microservo MG90S que sube y baja una uña abatible	DISEÑO
Fixture	21.3 × 16.5 cm de quilla y 6.4 cm de alto; seis por carga, a 7.0 cm de separación	DISEÑO
Batería	Plomo-ácido 12 V, 24 Ah, 8.3 kg, abajo y al centro, sujeta con correa	DISEÑO
Mesa de descarga	140 × 30 cm, inclinación ≈ 15°, borde a 83 cm	DISEÑO

Los planos técnicos de fabricación (cortes del perfil y piezas impresas) están en la hoja de armado; su exportación a plano acotado se lista en el Anexo B.

2.5 Diseño eléctrico: alimentación, protecciones, distribución y diagrama eléctrico

La Figura 9 es el esquema eléctrico de potencia del carrito, dibujado con símbolos normalizados (batería, fusibles, interruptores, relevador, motores, tierra). La batería alimenta dos ramas con su propio fusible: la de **motores** (F1 de 15 A: ruedas a 18 V y drivers de los motores a pasos a 12 V) pasa por el contacto de un relevador que controla el paro de emergencia, y la **auxiliar** (F2 de 10 A: Jetson, ESP32 y sensores) queda viva aunque se pulse el paro. El paro es de contacto normalmente cerrado y alimenta la bobina del relevador: al pulsarlo, el relevador cae y corta las ruedas y los motores a pasos a la vez. Los valores (fusibles, resistencias del divisor de batería, capacitores de 100 µF) son los de la hoja de armado rev. 3.8.

Decisión abierta: hoy el paro no está conectado a ningún pin del ESP32, por lo que el firmware no se entera de que fue pulsado. El equipo debe decidir si agrega esa señal (se propone un pin de solo lectura).

Celda. La Figura 10 muestra la conexión eléctrica de la celda. La fuente de 24 V del tablero alimenta el LOGO!, el infrarrojo y la electroválvula; la Raspberry Pi 4 tiene su propia fuente de 5 V. Ninguna señal de 24 V llega a un GPIO de la Raspberry: sus salidas GPIO 8 y GPIO 9 (3.3 V) activan un módulo de dos relevadores cuyos contactos secos llevan 24 V a la electroválvula del pistón y a la entrada I4 del LOGO!, que hace avanzar la banda. El infrarrojo entra a la entrada I3 del LOGO!. Las marcas y modelos del LOGO!, del infrarrojo y de la fuente se anotan en la lista de materiales.

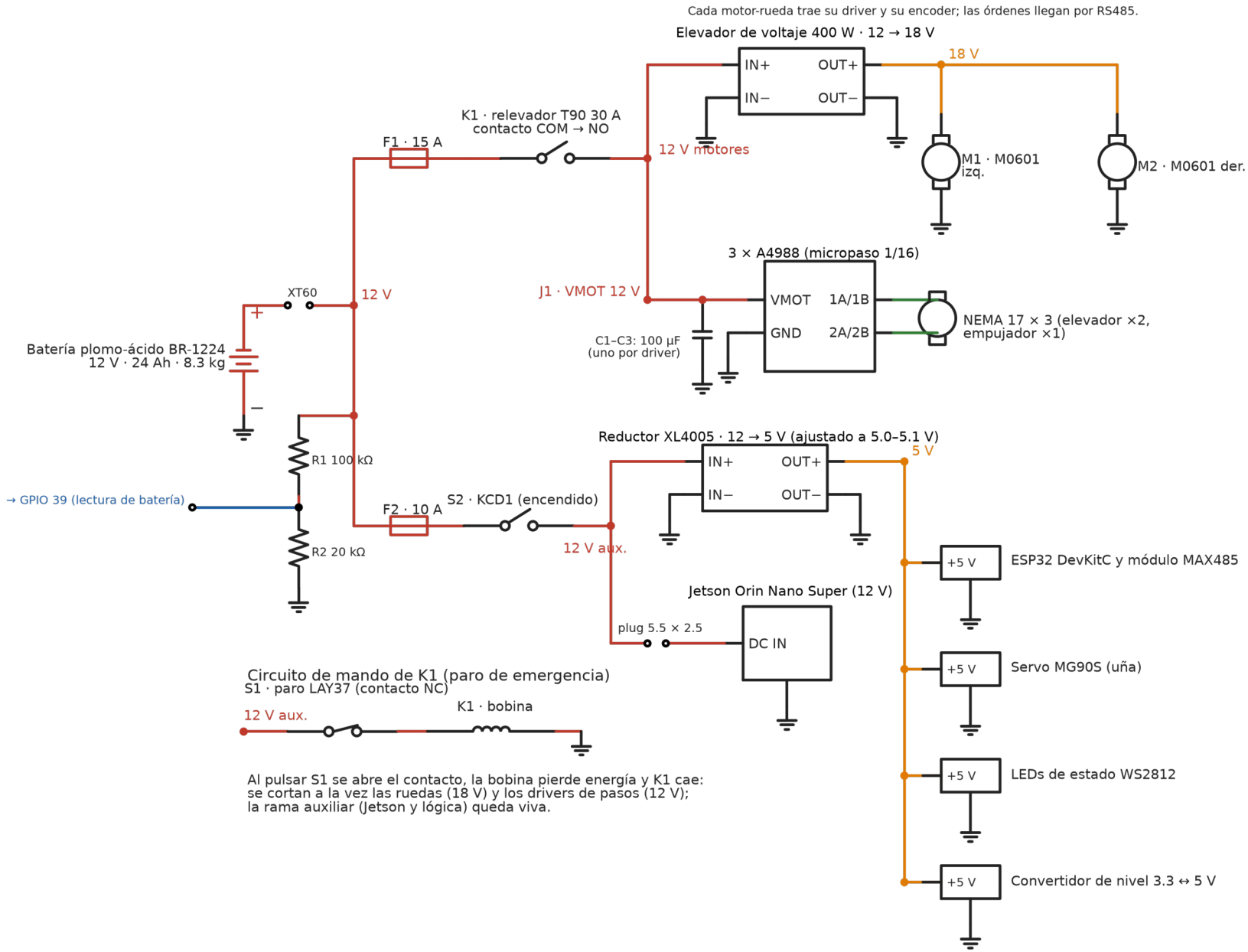
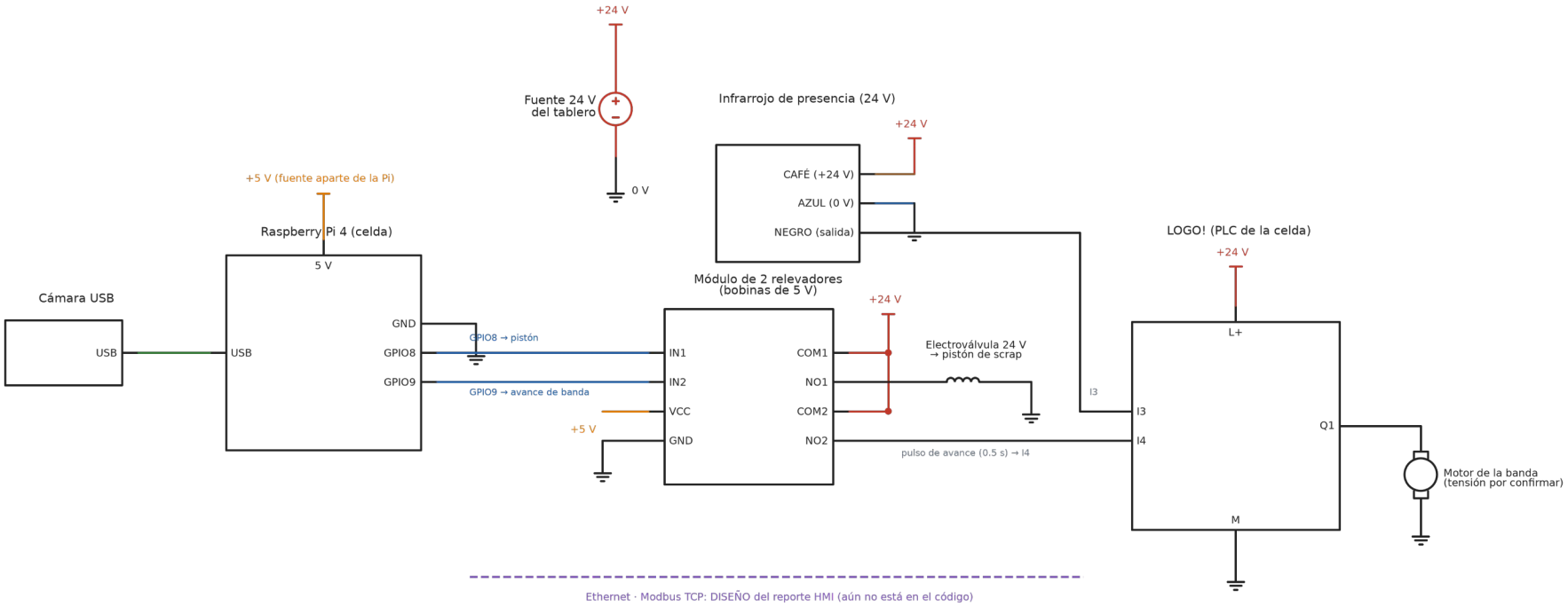


Figura 9. Esquema eléctrico de potencia y protecciones del carrito.



Ninguna señal de 24 V llega a un GPIO de la Raspberry Pi: los relevadores aíslan la lógica de 3.3 V de la de 24 V.

Figura 10. Esquema eléctrico de la celda: Raspberry Pi 4, relevadores, LOGO!, infrarrojo y electroválvula.

2.6 Diseño electrónico: sensores, actuadores, controladores, interfaces y comunicación

Tabla 10. Sensores, actuadores y controladores

Tipo	Elemento	Función	Interfaz
Controlador	Raspberry Pi 4	Visión de la celda y juicio PASS/FAIL	USB (cámara), GPIO, red
Controlador	LOGO! (PLC)	Secuencia de la banda; ignora su infrarrojo 5 s tras el pulso I4	Entradas I3 e I4, salida a motor
Controlador	Jetson Orin Nano Super	Visión del carrito, ruta y órdenes	USB al ESP32, red
Controlador	ESP32 DevKitC	Motores, sensores y watchdog	STEP/DIR, RS485, I²C, GPIO
Sensor	Cámara USB (celda) y Logitech 1080p (carrito)	Imagen para YOLO, QR y ArUco	USB
Sensor	Infrarrojo de presencia de 24 V	Detecta la pieza en la banda	Entrada I3 del LOGO!
Sensor	4 × VL53L0X	Distancia: frenan si algo está a menos de 40 cm	I²C (con XSHUT por PCF8574)
Sensor	IMU BNO055	Giro del carrito (corrige la odometría)	I²C
Sensor	Barra QTR-8A	Respaldo: seguir una línea en el piso	Entradas analógicas
Sensor	3 finales de carrera (NC)	Cero (home) del elevador y del empujador	GPIO
Actuador	Relevadores GPIO 8 y 9; electroválvula 24 V; pistón	Expulsar piezas FAIL y avanzar la banda	GPIO 3.3 V
Actuador	2 motores-rueda M0601	Mover el carrito	RS485 (MAX485)
Actuador	3 NEMA 17 con A4988; servo MG90S	Elevador, empujador y uña	STEP/DIR y PWM

La Figura 11 muestra la electrónica del carrito con sus cables en el modelo 3D. Las comunicaciones son: USB entre el Jetson y el ESP32; RS485 entre el ESP32 y los motores-rueda; I²C para los sensores; HTTP/JSON por la red local entre los agentes y el servidor; y, como diseño aún no implementado, Modbus TCP entre la Raspberry y el LOGO!.

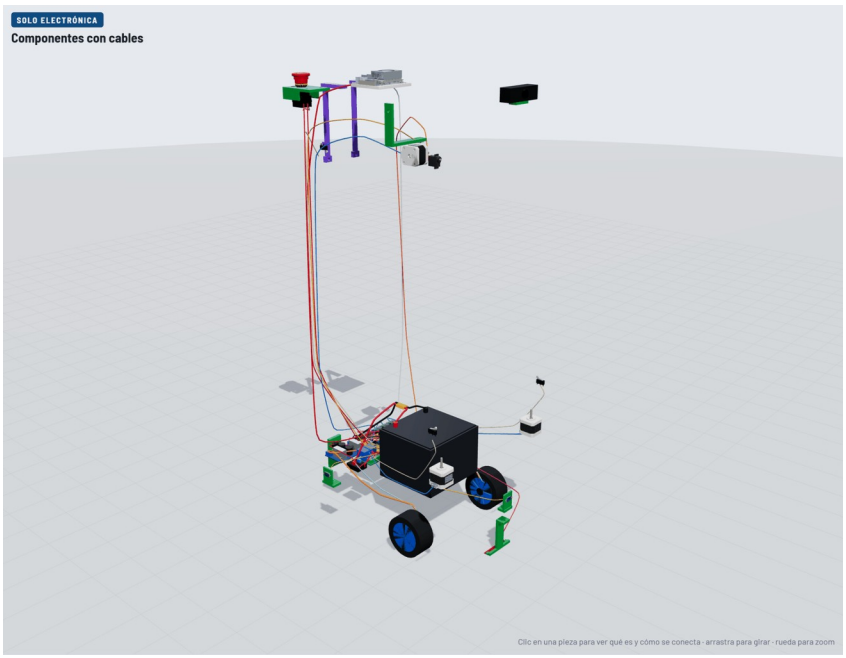


Figura 11. Modelo 3D de la electrónica con sus cables (vista «Solo electrónica»).

2.7 Diseño y programación del sistema de control

El control está repartido en capas, de la más lenta a la más rápida:

- **Servidor (laptop):** registra y muestra; no mueve hardware.
- **Jetson Orin Nano Super:** visión (QR, ArUco, YOLO), planeación de la ruta por waypoints (máximo 0.35 m/s y 0.4 m/s²) y órdenes al ESP32 por USB.
- **ESP32:** control de bajo nivel en tiempo real: pasos de los motores (micropaso 1/16, 3200 pulsos por vuelta), odometría con encoders e IMU, finales de carrera y un watchdog de 1 segundo que detiene todo si el Jetson deja de enviar órdenes. Se programará con PlatformIO (PENDIENTE).
- **Celda:** la Raspberry Pi 4 decide PASS/FAIL y activa dos relevadores; el LOGO! maneja la banda.

Tabla 11. Parámetros del ciclo de la celda (valores por defecto del código)

Parámetro	Valor	Estado
Periodo de análisis de cámara	0.5 s por cuadro	VERIFICADO en el código
Juicios iguales seguidos antes de actuar	5 (≈ 2.5 s)	VERIFICADO en el código
Umbral de confianza para PASS	0.50 (el detector trabaja con 0.25 internamente)	VERIFICADO en el código del agente
Espera antes del pistón (FAIL)	3 s	VERIFICADO en el código; calibrar en la banda real
Pistón activado (GPIO 8)	1 s	VERIFICADO en el código
Regreso del pistón	2 s	VERIFICADO en el código

Parámetro	Valor	Estado
Pulso de avance de la banda (GPIO 9 → entrada I4 del LOGO!)	0.5 s	VERIFICADO en el código
El LOGO! ignora su infrarrojo tras el pulso	5 s	DISEÑO (programa del LOGO!, por confirmar)

La Figura 12 muestra el tiempo de una pieza en la celda: aproximadamente 3 segundos si se acepta y 9 si se rechaza (sin contar el tiempo de cada análisis).

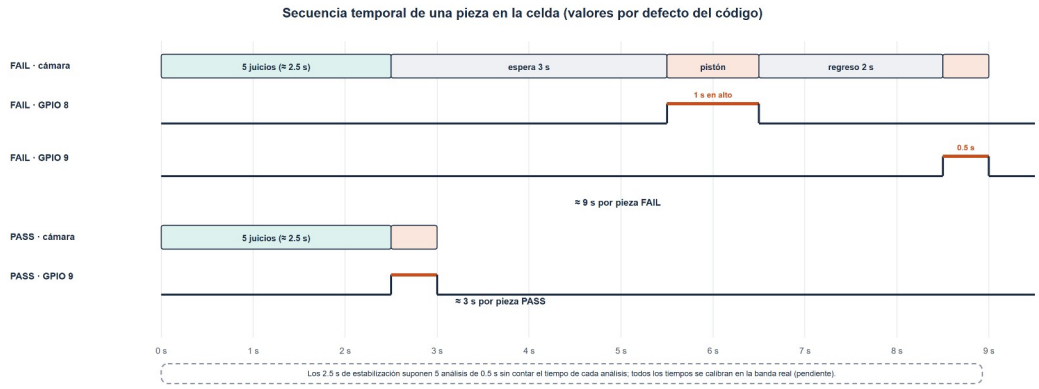


Figura 12. Diagrama de tiempos de la celda para una pieza aprobada y una rechazada.

2.8 Diagrama de conexiones / cableado

Las Figuras 13 y 14 son los esquemas de conexión del ESP32, con cada cable en su color. En la primera (13) se ven los tres drivers A4988 con sus motores NEMA 17 (dos mueven el elevador con las mismas señales STEP, DIR y ENABLE y uno el empujador), el servo MG90S de la uña y la tira de LEDs; las redes con nombre (A_STEP12, A_DIR12, A_EN, etc.) unen cada pin del ESP32 con su driver, como en un esquema de Proteus. En la segunda (14) se ven el módulo MAX485 con su convertidor de nivel hacia los dos motores-rueda por RS485, los tres finales de carrera normalmente cerrados, la barra QTR-8A, el bus I²C con el expansor PCF8574, la IMU y los cuatro sensores VL53L0X (cuyos pines XSHUT salen del PCF8574 para darles direcciones distintas) y la lectura de la batería por divisor de voltaje. Los pines GPIO 34, 35, 36 y 39 solo leen.

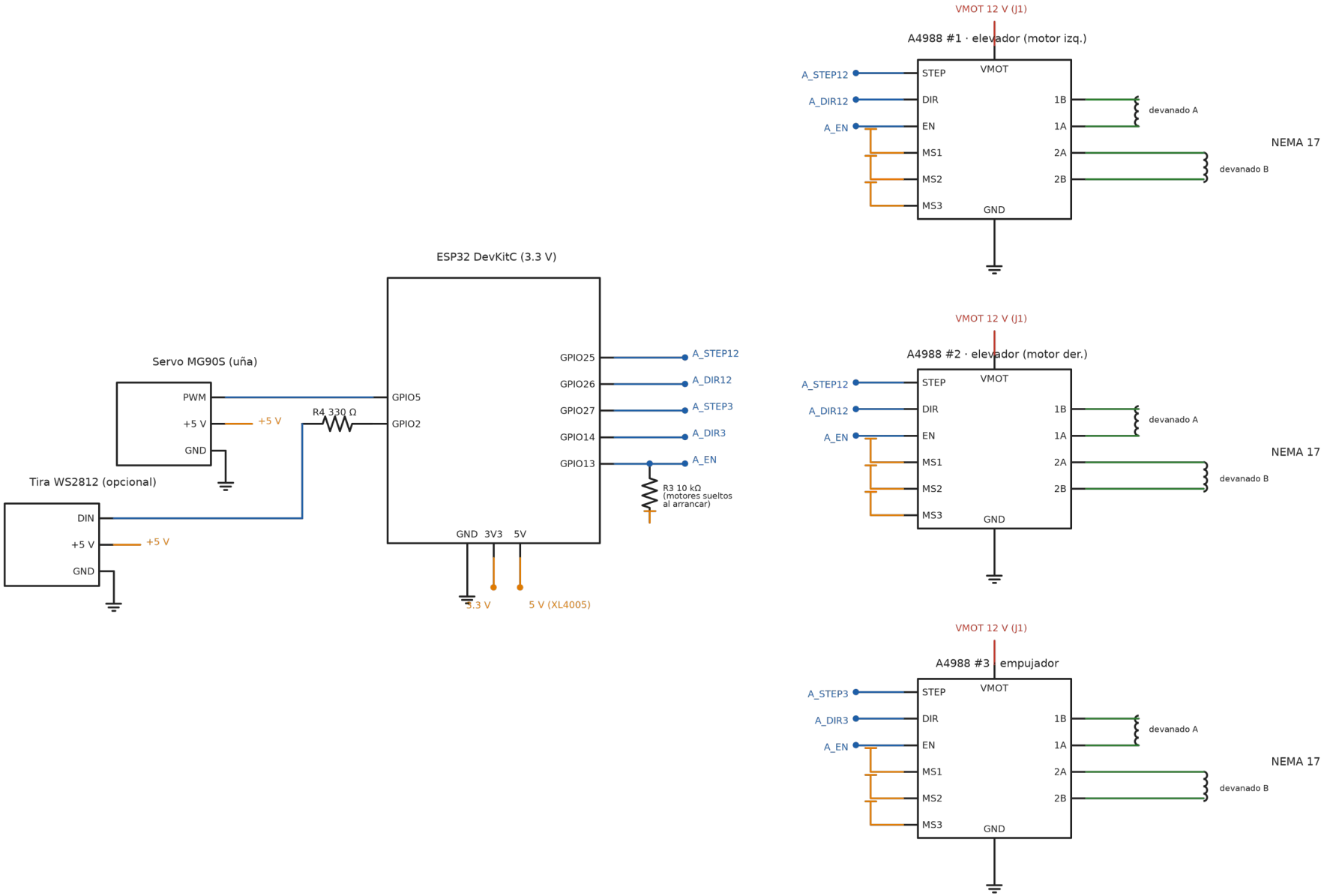


Figura 13. Conexiones del ESP32 (1 de 2): motores a pasos, servo y LEDs.

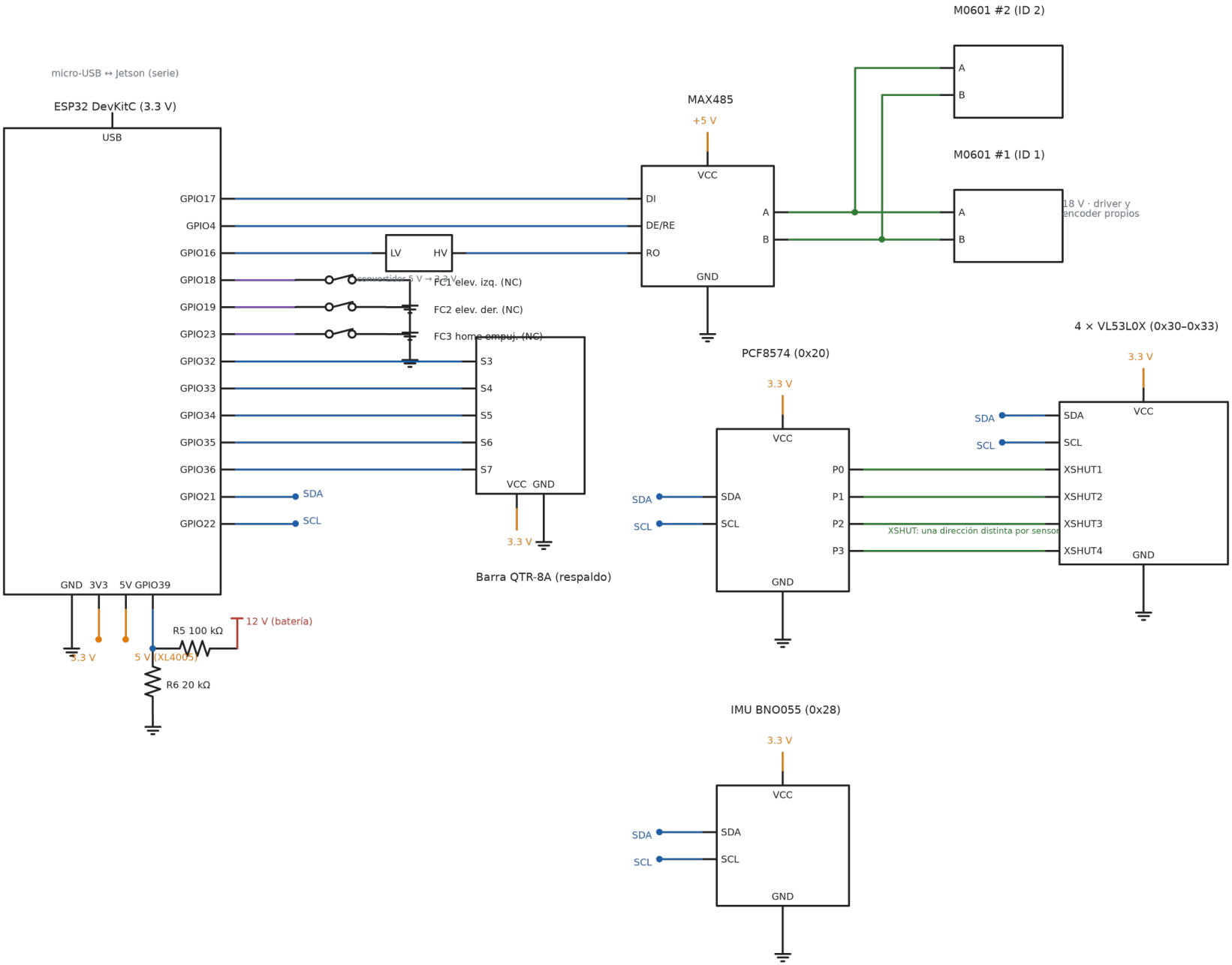


Figura 14. Conexiones del ESP32 (2 de 2): RS485, finales de carrera, QTR-8A, I²C y lectura de batería.

La placa perforada de 9 × 15 cm lleva 107 cables en 14 grupos; cada uno tiene color, calibre y largo en la tabla de cables de la hoja de armado. La conexión eléctrica de la celda (infrarrojo → LOGO!, salidas GPIO 8 y 9 → relevadores) está en la Figura 10.

2.9 Programas y algoritmos principales

Se explican los cuatro algoritmos más importantes. El código completo está en los Anexos A y F.

2.9.1 Juicio estable de la celda

Cada 0.5 s la cámara entrega un juicio (PASS, FAIL o SIN_PIEZA). El programa cuenta cuántos juicios iguales van seguidos; solo cuando llegan a cinco y no está ocupada, dispara la secuencia una sola vez por pieza y se «desarma» hasta ver SIN_PIEZA, para no repetir la acción sobre la misma pieza. Fragmento (cell_line.py, método ver):

```
def ver(self, juicio):
    with self._lock:
        if juicio == self._ultimo:
            self._racha += 1          # otro juicio igual seguido
        else:
            self._ultimo, self._racha = juicio, 1
        if self._busy or not self._running or self._racha < self.estables:
            return                    # aún no hay 5 iguales
        if juicio == "SIN_PIEZA":
            self._armed = True        # se permite una pieza nueva
            return
        if juicio in ("PASS", "FAIL") and self._armed:
            self._armed, self._busy = False, True   # una sola vez
```

2.9.2 Secuencia del pistón y de la banda

Si el juicio es FAIL: espera 3 s a que la pieza llegue al pistón, enciende el GPIO 8 durante 1 s, lo apaga y espera 2 s a que regrese; después pulsa el GPIO 9 durante 0.5 s para que el LOGO! avance la banda. Si es PASS solo se hace el pulso del GPIO 9. Si no hay chip GPIO, el sistema no finge: anota «fallo_actuacion» (FAIL) o «pulso_no_enviado» (PASS). Un paro del operador aborta la secuencia y apaga ambas salidas.

2.9.3 Envío de observaciones en lotes

El agente del Jetson no envía un POST por cada observación: las junta y las manda cada cierto intervalo (variable AETHER_OUTBOX_INTERVAL_S). Cada observación lleva un identificador único (event_id); si el servidor la recibe dos veces, descarta la repetida, de modo que un reintento por falla de red no duplica el inventario.

2.9.4 Navegación y ciclo del carrito (diseño)

El Jetson planea una ruta por waypoints con odometría de los encoders corregida por la IMU; los sensores de distancia frenan si algo aparece a menos de 40 cm; el embudo y el ArUco centran el carrito en cada estación (±5 mm). El ESP32 ejecuta las órdenes y se detiene solo si pasa 1 s sin recibir ninguna. Este algoritmo es diseño: no existe aún el firmware.

2.10 Lista de materiales, cantidades, marca/modelo, costo y fuente de adquisición

La lista incluye **todos los componentes de la hoja de armado del carrito (revisión 3.8)**, los materiales adicionales de la hoja de costos y los de la celda. Las columnas amarillas (marca o modelo, fuente de adquisición y costos) las llena el equipo con sus cotizaciones: la hoja de costos que se tiene (PDF) no conserva las columnas de precio y no se inventó ninguno. Del motor-rueda M0601 se decidió el modelo y ya existe cotización; su precio debe capturarse aquí. Donde la hoja de armado ya nombra el modelo, este aparece en el nombre del componente.

Tabla 12. Lista de materiales del carrito y la estación (69 componentes de la hoja de armado)

N.º	Componente	Cantidad	Subsistema	Marca / modelo	Fuente de adquisición	Costo unitario (MXN)	Costo total (MXN)
1	Poste de perfil 3030 · 950 mm	4	Estructura				
2	Larguero · 440 mm	4	Estructura				
3	Travesaño de marco · 340 mm	4	Estructura				
4	Travesaño central · 340 mm	1	Estructura				
5	Columna de la torre · 890 mm	4	Estructura				
6	Travesaño superior central · 340 mm	1	Estructura				
7	Travesaño del empujador · 198 mm	1	Estructura				
8	Escuadras impresas (PETG)	28	Estructura				
9	Placas de unión impresas	16	Estructura				
10	Tuerca T de martillo + Allen M6	100 juegos	Estructura				
11	Panel frontal MDF 3 mm	1	Estructura				
12	Rodaja twin de nylon 50 mm	4	Tracción				
13	Placa pivotante de la suspensión	2	Tracción				
14	Resorte de compresión	2	Tracción				
15	Motor-rueda M0601	2	Tracción				
16	Cuna impresa de la batería	1	Energía				
17	Batería BR-1224 (12 V, 24 Ah)	1	Energía				

N.º	Componente	Cantidad	Subsistema	Marca / modelo	Fuente de adquisición	Costo unitario (MXN)	Costo total (MXN)
18	Correa con hebilla (o cincho de trinquete)	1	Energía				
19	Guía en L impresa	4	Torre y empujador				
20	Soporte inferior (NEMA + KFL08)	2	Torre y empujador				
21	Soporte superior (KFL08)	2	Torre y empujador				
22	NEMA 17 del elevador (17HS4401)	2	Torre y empujador				
23	Cople flexible 5 → 8 mm	2	Torre y empujador				
24	Chumacera KFL08	4	Torre y empujador				
25	Husillo TR8 · 60 cm	2	Torre y empujador				
26	Varilla lisa 8 mm × 60 cm	2	Torre y empujador				
27	Rodamiento lineal SC8UU	6	Torre y empujador				
28	Tuerca T8 de latón	2	Torre y empujador				
29	Plataforma impresa 21.3 × 16.5 cm	1	Torre y empujador				
30	Final de carrera (contacto NC)	3	Torre y empujador				
31	Varilla 8 mm × 30 cm (empujador)	2	Torre y empujador				
32	Soportes delanteros del empujador	1 juego	Torre y empujador				
33	Soporte trasero y final de home	1	Torre y empujador				
34	Carro, placa 15 × 3 cm y cremallera	1	Torre y empujador				
35	Piñón m1.5 · 20 dientes	1	Torre y empujador				
36	NEMA 17 del empujador	1	Torre y empujador				
37	Microservo MG90S (180°)	1	Torre y empujador				
38	Brazo del servo	1	Torre y empujador				
39	Uña abatible (dedo impreso de 7 mm)	1	Torre y empujador				
40	Fixture con ECU (quilla + 2 bastidores)	6	Torre y empujador				

N.º	Componente	Cantidad	Subsistema	Marca / modelo	Fuente de adquisición	Costo unitario (MXN)	Costo total (MXN)
41	Bandeja de electrónica	1	Electrónica y potencia				
42	Placa perforada 9 × 15 cm	1	Electrónica y potencia				
43	Tiras header hembra (zócalos)	~120 pines	Electrónica y potencia				
44	ESP32 DevKitC (38 pines)	1	Electrónica y potencia				
45	Driver A4988	3	Electrónica y potencia				
46	Capacitor electrolítico 100 µF	3	Electrónica y potencia				
47	Convertidor de nivel 3.3 ↔ 5 V (4 canales)	1	Electrónica y potencia				
48	Módulo MAX485 (RS485)	1	Electrónica y potencia				
49	Expansor PCF8574	1	Electrónica y potencia				
50	IMU BNO055	1	Electrónica y potencia				
51	Resistencias y capacitor de 100 nF	4 + 1	Electrónica y potencia				
52	Borneras KF301 y header del servo	12	Electrónica y potencia				
53	Elevador de voltaje 400 W (12 → 18 V)	1	Electrónica y potencia				
54	Reductor XL4005 (12 → 5 V)	1	Electrónica y potencia				
55	Módulo relé T90 30 A	1 (2 canales, se usa 1)	Electrónica y potencia				
56	Conector XT60 (hembra + macho)	1 par	Electrónica y potencia				
57	Portafusibles 15 A y 10 A	2	Electrónica y potencia				
58	Interruptor KCD1	1	Electrónica y potencia				
59	Paro de emergencia LAY37 (NC)	1	Electrónica y potencia				
60	Sensor de distancia VL53L0X	4	Navegación y cómputo				
61	Barra IR QTR-8A	1	Navegación y cómputo				
62	Placa del Jetson (MDF)	1	Navegación y cómputo				
63	Jetson Orin Nano Super	1	Navegación y cómputo				

N.º	Componente	Cantidad	Subsistema	Marca / modelo	Fuente de adquisición	Costo unitario (MXN)	Costo total (MXN)
64	Cámara Logitech 1080p	1	Navegación y cómputo				
65	Banda transportadora (estación de carga)	1	Estación				
66	Mesa inclinada de descarga	1	Estación				
67	Mesa de trabajo	—	Herramientas				
68	Nivel de burbuja	1	Herramientas				
69	Sobra de perfil	—	Herramientas				

Tabla 13. Materiales adicionales de la hoja de costos que no figuran como componente de la hoja de armado

N.º	Componente	Cantidad	Subsistema	Marca / modelo	Fuente de adquisición	Costo unitario (MXN)	Costo total (MXN)
1	Cargador inteligente BR-700 (6 A) para plomo-ácido	1	Electrónica y potencia				
2	Cable de silicona flexible 14 AWG (3 m rojo y 3 m negro)	6 m	Electrónica y potencia				
3	Tira Neopixel WS2812 5050 (LEDs de estado)	3	Electrónica y potencia				
4	Fusible automotriz 15 A (uno más de repuesto)	2	Electrónica y potencia				
5	Fusible automotriz 10 A (uno más de repuesto)	2	Electrónica y potencia				
6	Faston hembra 1/4 in con forro	2	Electrónica y potencia				
7	Terminales de ojillo para 12–14 AWG (bornes de la batería)	1 juego	Electrónica y potencia				
8	Plug DC 5.5 × 2.5 mm con cable (alimentación del Jetson)	1	Electrónica y potencia				
9	Cámara Yahboom IMX219 8 MP 77° (CSI), planeada para el Jetson	1	Navegación y cómputo				
10	Marcadores ArUco, embudos de acople e impresos de apoyo	1 juego	Navegación y cómputo				
11	Perfil 3030 faltante (por definir con el corte final)	1	Estructura				

N.º	Componente	Cantidad	Subsistema	Marca / modelo	Fuente de adquisición	Costo unitario (MXN)	Costo total (MXN)
12	Escuadras para 3030 impresas en PETG (16 de 36 × 36 × 20 y 12 de otra medida)	28 (la hoja de costos anota 50)	Estructura				
13	Varilla lisa de 8 mm × 60 cm (segunda guía de la plataforma)	1	Torre y empujador				

Tabla 14. Tornillería y consumibles (hoja de costos)

N.º	Componente	Cantidad	Subsistema	Marca / modelo	Fuente de adquisición	Costo unitario (MXN)	Costo total (MXN)
1	Tuerca T de martillo M6 (drop-in)	confirmar	Sujetan todo al perfil de 8 mm de ranura				
2	Tornillo M6 × 12 cabeza Allen	confirmar	Escuadras (56), placas y piezas de 5–6 mm				
3	Tornillo M6 × 16 cabeza Allen	18	Guías traseras de la pila y piezas de 8 mm				
4	Rondana plana M6	confirmar	Bajo cada cabeza Allen M6				
5	Tornillo M6 × 20 cabeza Allen	4	Bandeja de electrónica				
6	Tornillo M6 × 12 cabeza plana avellanada	12	Guías delanteras de la pila				
7	Tornillo M8 × 40 + tuercas M8 (pivote y precarga) + rondana M8	2 + 2 + 2 + 12	Pivote de la suspensión				
8	Tornillo M8 × 70 + tuerca T M8	2	Ranura del larguero				
9	Tornillo M3 × 8	24	3 NEMA 17 y separadores de la placa				
10	Tornillo M3 × 10	8	4 sensores VL53L0X (2 por sensor)				
11	Tornillo M3 × 12	10	Elevador de voltaje y relevador a la bandeja				
12	Tornillo M3 × 20	10	Tuercas T8 de los husillos				
13	Tuerca hexagonal M3	32	Varias uniones				
14	Prisionero M3 × 12	2	Piñón del empujador				

N.º	Componente	Cantidad	Subsistema	Marca / modelo	Fuente de adquisición	Costo unitario (MXN)	Costo total (MXN)
15	Tornillo M4 × 12	24	6 rodamientos SC8UU				
16	Tornillo M4 × 20 + tuerca	4	Placa de la plataforma				
17	Tornillo M2.5 × 12 autorroscante	6	3 finales de carrera				
18	Tornillo M2.5 × 12 cabeza plana avellanada	8	3 por M0601 a la placa pivotante y 2 de otra pieza				
19	Tornillo M5 × 20 + tuerca y rondana	8	Chumaceras KFL08				
20	Tornillo M2 × 8 + tuerca	2	Barra QTR-8A				
21	Tornillería de la cremallera y del soporte	1 juego	Cremallera del empujador				
22	Separadores M3 de nylon (20) y lubricante	1 juego	Montar la placa en la bandeja y lubricar guías				
23	Fijador de roscas Loctite azul 243	1	Tornillos del M0601 y pivotes				

Tabla 15. Componentes de la celda de manufactura (por confirmar con el equipo de la celda)

N.º	Componente	Cantidad	Subsistema	Marca / modelo	Fuente de adquisición	Costo unitario (MXN)	Costo total (MXN)
1	Raspberry Pi 4	1	Cómputo de la celda: cámara, YOLO ONNX, GPIO				
2	Cámara USB de la celda	1	Imagen para el juicio PASS/FAIL				
3	LOGO! (controlador)	1	Maneja la banda; entrada I4 (avance) e I3 (infrarrojo)				
4	Módulo de 2 relevadores (GPIO 8 y GPIO 9)	1	Pistón y avance de la banda; ninguna señal de 24 V llega a un GPIO				
5	Electroválvula de 24 V y pistón	1	Expulsa las piezas FAIL				
6	Sensor infrarrojo de presencia (24 V)	1	Entrada I3 del LOGO!				

N.º	Componente	Cantidad	Subsistema	Marca / modelo	Fuente de adquisición	Costo unitario (MXN)	Costo total (MXN)
7	Banda transportadora (3 tramos de 1030 mm)	1	Cinta de 20.35 cm de ancho				
8	Fuente de 24 V y de 5 V del tablero	confirmar	Alimentación separada de LOGO!, sensores y Pi				
9	Contenedor de scrap, switch de red y cables	confirmar	Recibe piezas rechazadas; red local				

Total estimado del proyecto (MXN): por calcular al llenar los costos. Este documento no repite precios que no pudo leer en la hoja de costos.

2.11 Distribución física / layout / boceto de maqueta

La Figura 15 es una vista superior. Están dibujados **a escala** la banda (309 cm), el carrito (50 × 40 cm) y la mesa de descarga (140 × 30 cm). La posición de la cámara, el infrarrojo, el pistón, el contenedor de scrap y el tablero a lo largo de la banda es **esquemática** y sus cotas quedan pendientes hasta medir la banda armada.

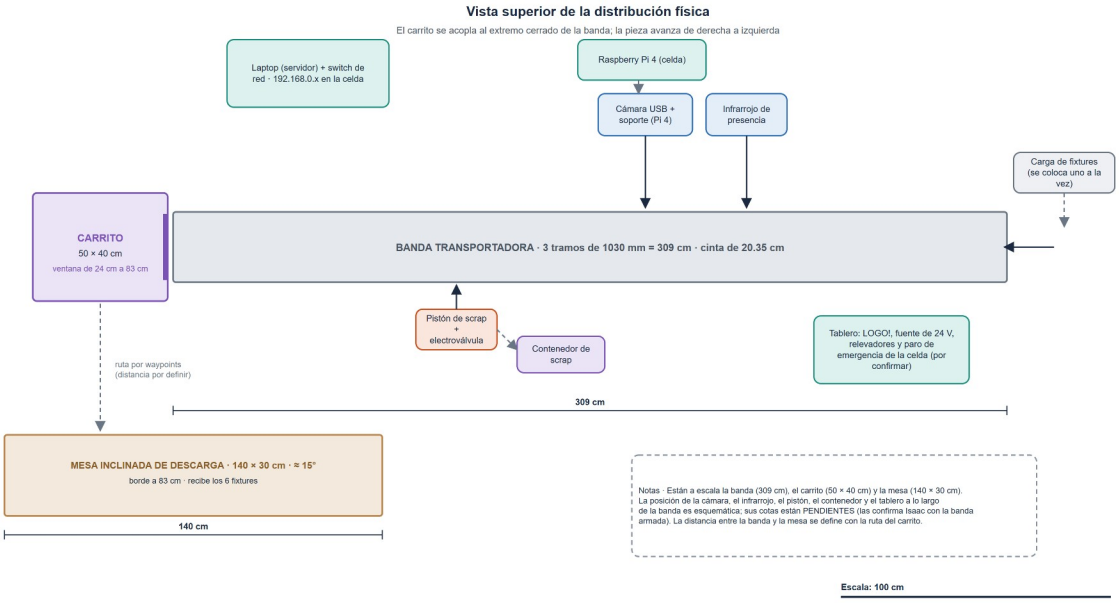


Figura 15. Distribución física de la celda, el carrito y la mesa de descarga (vista superior).

2.12 Procedimiento de construcción, integración y puesta en marcha

Esta sección separa lo que **ya existe y se vio funcionar**, lo que **existe como diseño** y lo que **falta**. Las fotos las toma el equipo después y se pegan en los recuadros indicados.

Tabla 16. Estado de cada parte

Parte	Estado	Evidencia
Servidor, base de datos y panel web	Implementado	Código en el repositorio y pruebas automáticas; capturas en la sección 4.7
Agente de la celda (visión, juicio, GPIO)	Implementado; prueba completa con la banda PENDIENTE	Código cell_line.py y celda_agent.py; pruebas de cell_line pasan
Pistón con el LOGO! de la celda	Funciona (informado por el equipo)	foto o video por agregar
Jetson Orin Nano Super	Puesto en marcha: PyTorch de NVIDIA funciona	Bitácora de hardware real del repositorio (21 de octubre de 2026)
Asistente de voz Lumina	Implementado (activación por palabra, texto vía Vosk)	Código y pruebas

Parte	Estado	Evidencia
Diseño 3D del carrito y hoja de armado	Terminado (rev. 3.8)	Figuras 5 a 8
Construcción física del carrito	PENDIENTE	—
Firmware del ESP32	PENDIENTE (se hará con PlatformIO)	—
Enlace Modbus TCP entre la Pi y el LOGO!	PENDIENTE (diseño del reporte HMI)	—

2.12.1 Celda de manufactura

El controlador de la celda es el LOGO! y ya funciona con el pistón. Esta subsección la redacta el equipo de la celda: se deja en blanco a propósito para que escriba cómo se construyó, cómo se programó el LOGO!, cómo se conectó el pistón y qué se probó.

ESCRIBIR AQUÍ: construcción e implementación del PLC (LOGO!) y del pistón — por el equipo de la celda

Cómo se cableó el LOGO! y qué entradas y salidas usa
Programa del LOGO! (diagrama de funciones o de escalera)
Conexión del pistón, la electroválvula y los relevadores
Pruebas realizadas y resultado



Figura 16. Vista general de la celda armada.



Figura 17. Tablero con el LOGO!, la fuente de 24 V y los relevadores.

ESPACIO PARA FOTO O CAPTURA
Foto del pistón montado junto a la banda.

Figura 18. Pistón de scrap y su electroválvula.

2.12.2 Carrito (Aether Inventory)

El carrito está **diseñado, documentado y con la lista de materiales lista**, pero no se ha construido. El orden de armado previsto en la hoja de armado es: marco inferior y superior con postes y largueros; suspensión y motores-rueda; batería y su correa; columnas y torre del elevador; empujador y uña; bandeja de electrónica y cableado; Jetson, sensores y panel frontal. Al empezar el armado, el equipo agrega las fotos de cada etapa.

ESPACIO PARA FOTO O CAPTURA
Foto del avance de la estructura cuando se arme.

Figura 19. Perfiles de 3030 cortados y marco armado.

ESPACIO PARA FOTO O CAPTURA
Foto de la placa perforada con ESP32, drivers y módulos.

Figura 20. Bandeja de electrónica armada en la mesa.

2.12.3 Software: prueba local en la laptop

El software de la celda se puede probar en la laptop sin la Raspberry Pi ni el pistón: el servidor del repositorio (main.py) ya trae el modo local, en el que la cámara de la laptop alimenta el juicio PASS/FAIL, y el programa de la celda (cell_line.py) sigue la misma secuencia aunque no haya chip GPIO. Para las capturas siguientes se arrancó el servidor local con las salidas GPIO **simuladas** y juicios PASS/FAIL **inyectados** cada 0.5 s (no hay cámara de celda ni banda en esta prueba), y se inició sesión en la pantalla de la celda con una cuenta de prueba local. Los conteos y las últimas piezas de la captura son de esa prueba, no de la banda real. Lumina se capturó en reposo («Dormida»).

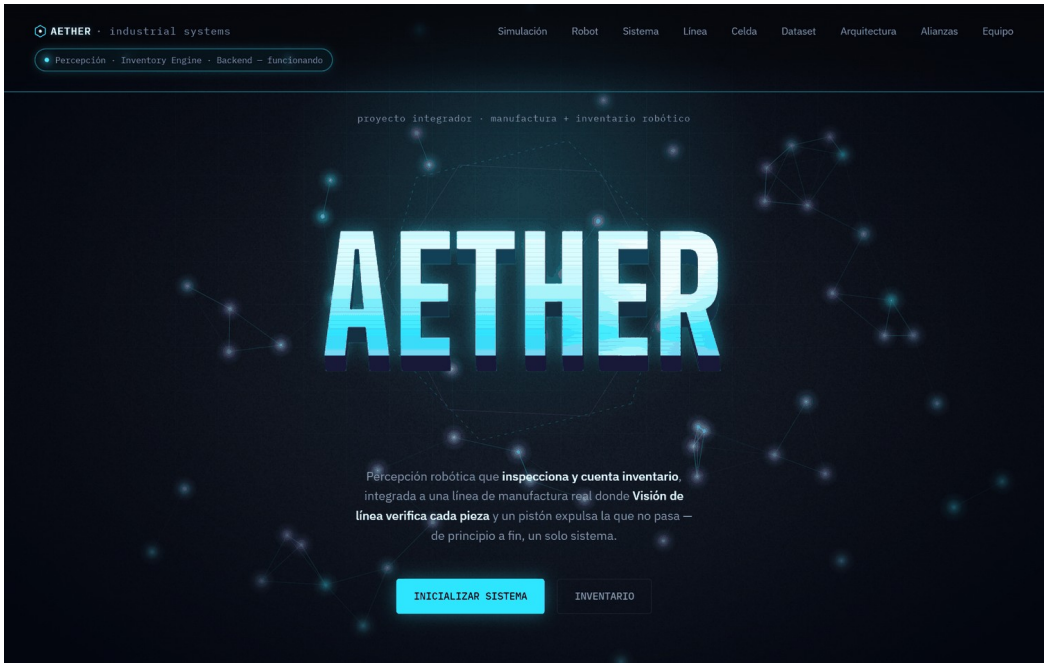


Figura 21. Portada del panel web (servidor local).

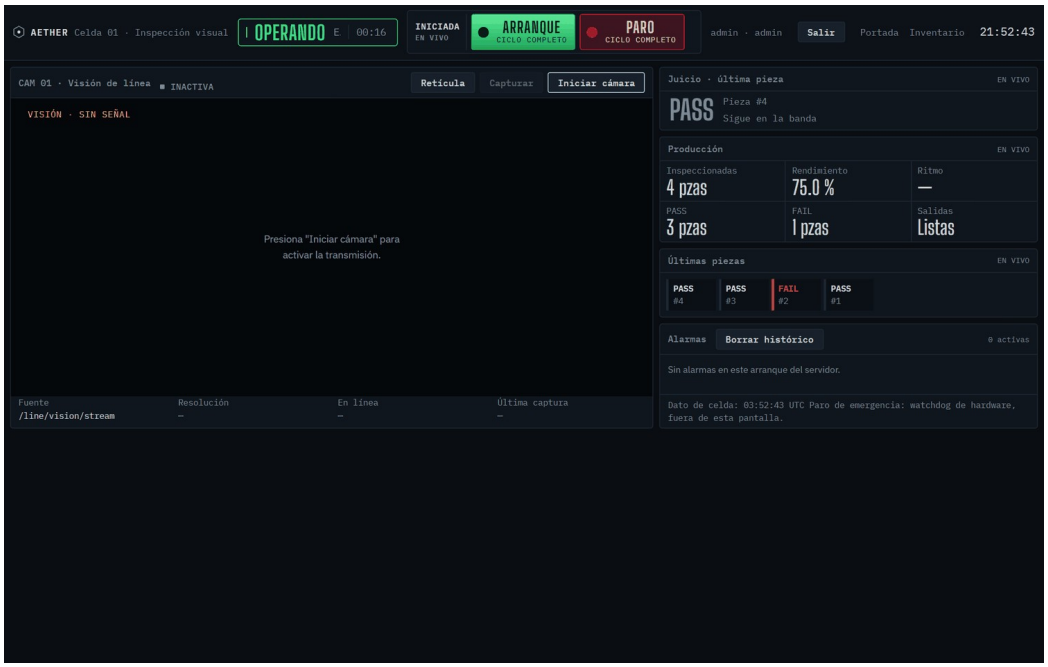


Figura 22. Pantalla de la celda (celda.html) en prueba local: estado «Operando», juicio, producción, últimas piezas y alarmas; GPIO simulado.

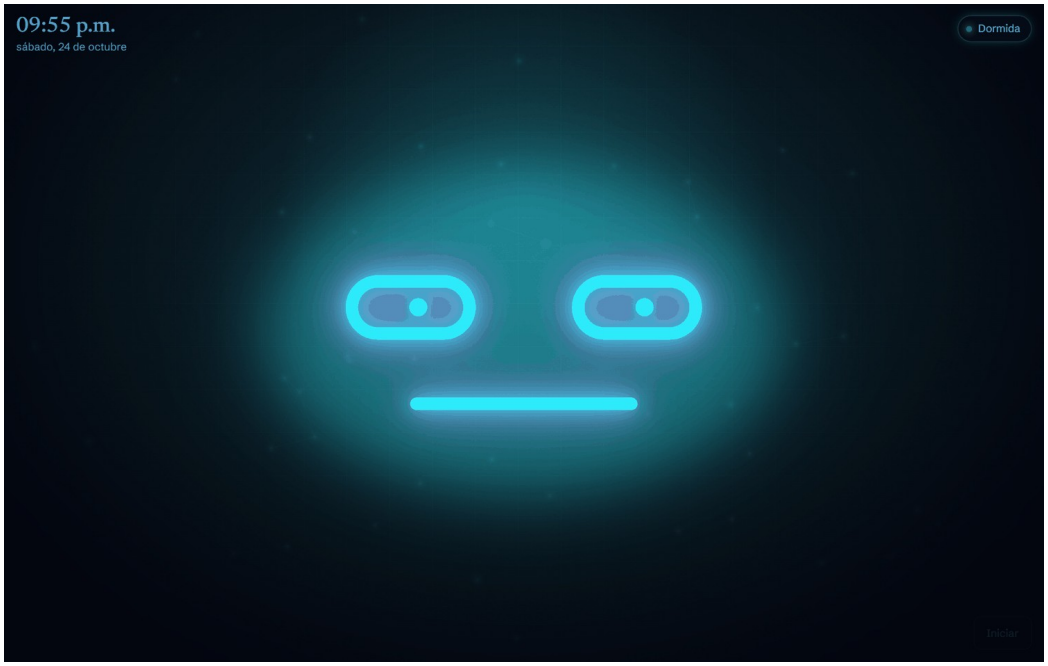


Figura 23. Lumina, el asistente del robot, en reposo (robot.html).

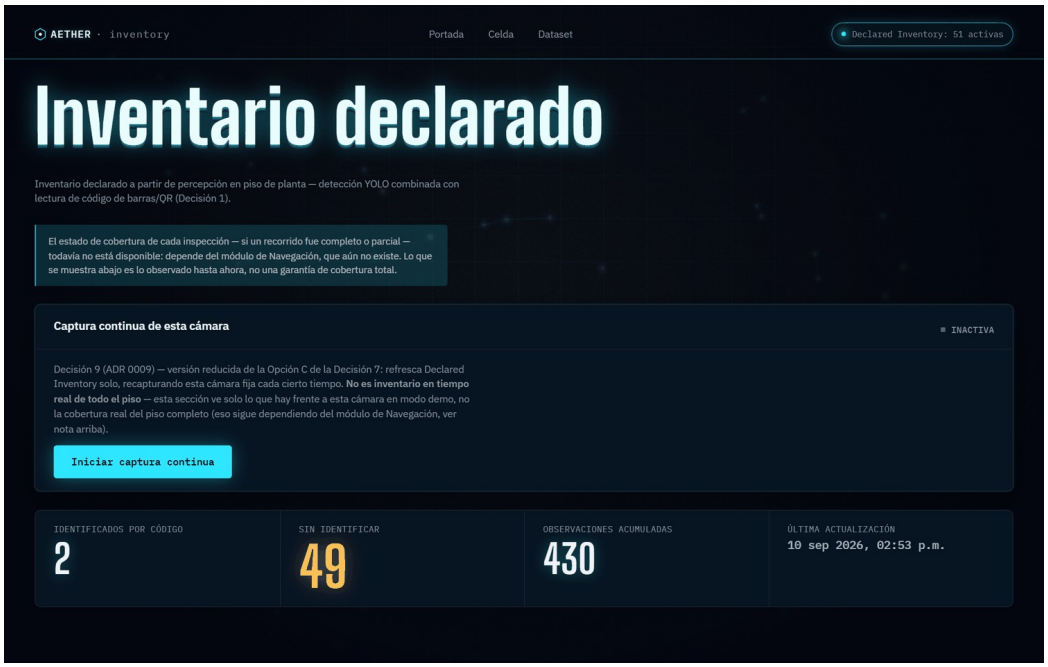


Figura 24. Inventario declarado (inventory.html).

2.13 Pruebas preliminares, fallas encontradas y ajustes realizados

Se separan las pruebas **hechas** de las **por hacer**. Para las segundas se indica qué se mide, cómo y con qué criterio se aprueba; los criterios son una **propuesta** que el equipo puede ajustar.

Tabla 17. Pruebas preliminares

N.º	Qué se prueba	Cómo	Criterio de aceptación	Estado y resultado
1	Lógica del servidor, los agentes y la celda	pytest sobre tests/backend (24 de octubre de 2026, laptop con Windows)	Todas pasan	VERIFICADO: 285 pasan y 11 fallan, todas con el mismo error de Windows al lanzar un subproceso (WinError 6, parece del entorno de ejecución). test_main.py no se pudo ejecutar porque falta la librería sqlalchemy en ese entorno
2	Puesta en marcha del Jetson	Importar PyTorch de NVIDIA y abrir la cámara	Sin errores; cámara con imagen	HECHA (bitácora del repositorio, 21 de octubre de 2026)
3	Pistón con el LOGO!	Activar el pistón con la salida de la celda	El pistón sale y regresa en cada orden	HECHA según el equipo; agregar registro o video
3a	Pantalla y secuencia de la celda en la laptop	Servidor local con GPIO simulado y juicios inyectados (24 de octubre de 2026)	La pantalla muestra «Operando», cuenta PASS y FAIL y completa la secuencia del pistón	VERIFICADO a la vista: la pantalla contó 3 PASS y 1 FAIL con el estado «Operando»; sin hardware real (Figura de la sección 2.12.3)
4	Precisión del modelo YOLO	Conjunto de piezas de validación del dataset	mAP50 ≥ 0.90	Reportada en el entrenamiento (0.995); no se repitió en esta revisión
5	Juicio con la banda real	20 piezas buenas y 20 malas por la banda	≥ 95 % bien clasificadas y 0 fallos de expulsión	PENDIENTE

N.º	Qué se prueba	Cómo	Criterio de aceptación	Estado y resultado
6	Tiempos del ciclo FAIL y PASS	Cronometrar 10 ciclos	Cada pieza FAIL expulsada y la banda reanuda sin atascos	PENDIENTE (calibrar la espera de 3 s)
7	Falla del pistón	Desconectar el relevador y pasar una pieza FAIL	El sistema registra «fallo_actuacion» y levanta la alarma	PENDIENTE
8	Enlace Modbus TCP Pi ↔ LOGO!	Leer y escribir los registros y esperar el acuse	Acuse recibido en menos de 1 s	PENDIENTE (aún sin implementar)
9	Paro de emergencia del carrito	Pulsar el paro con motores energizados y medir 12 V y 18 V	0 V en ruedas y drivers; Jetson sigue encendido	PENDIENTE (carrito sin construir)
10	Tensión de la fuente de 5 V	Medir con multímetro antes de conectar el ESP32	5.0 a 5.1 V	PENDIENTE
11	Elevador de seis niveles	Subir y bajar con seis fixtures	Paso de 7.0 cm ± 1 mm y sin atoramiento	PENDIENTE
12	Acople a la banda	Acoplar 10 veces con el embudo y el ArUco	Error ≤ 5 mm en las 10 veces	PENDIENTE
13	Frenado por obstáculo	Colocar un obstáculo a menos de 40 cm	El carrito se detiene antes de tocarlo	PENDIENTE
14	Ciclo completo celda → carrito → mesa	Un fixture de la banda a la mesa	Fixture en la mesa y observación registrada en el servidor	PENDIENTE

Tabla 18. Fallas encontradas y ajustes realizados

N.º	Problema	Causa	Ajuste	Estado
1	Los brazos robóticos no caben en tiempo y equipo disponibles	Alcance demasiado grande	ADR 0010: banda con fixtures y pistón de expulsión (16 de septiembre de 2026)	Resuelto
2	La Raspberry Pi 5 no es necesaria	El Jetson cubre el robot	Se descartó la Pi 5; la celda usa una Pi 4	Resuelto
3	PyTorch no corría en el Jetson	Faltaba una librería de NVIDIA (libcusparseLt); numpy 2 y un paquete que arrastraba torch	Se instaló la librería, se fijó numpy menor que 2 y se quitó el paquete	Resuelto (bitácora)
4	El código fallaba en Python 3.10 del Jetson	Se usaba datetime.UTC	Se cambió a timezone.utc	Resuelto
5	El video del agente quedaba bloqueado en el panel	La política de seguridad del navegador (CSP) no incluía el origen del agente	Se agregó el origen; el mensaje de error distingue «bloqueado por seguridad» de «cámara no disponible»	Resuelto
6	El audio de la voz de Lumina no sonaba	La CSP no permitía audio de tipo blob y data	Se agregó media-src con blob: y data:	Resuelto
7	Vosk no conoce la palabra «lumina»	No está en su vocabulario	Palabra de activación tolerante y emisión por fin de frase	Resuelto
8	Un POST por cada observación saturaba el servidor	Envío uno a uno	Las observaciones se juntan en lotes (AETHER_OUTBOX_INTERVAL_S)	Resuelto
9	Navegadores del Jetson	Problema abierto anotado en la bitácora	Opciones en el ADR 0011	Abierto
10	El paro de emergencia no avisa al ESP32	El diseño solo corta el relevador	Decidir si se agrega un pin de lectura	Abierto

N.º	Problema	Causa	Ajuste	Estado
11	La hoja de costos no conserva las columnas de precio	Se exportó sin esas columnas	El equipo captura costos en la lista de materiales	Pendiente
12	El enlace Modbus TCP del reporte HMI no está en el código	Solo está en el diseño	Implementarlo y probarlo antes de declararlo	Pendiente

Capítulo 3. Administración y dirección

3.1 Plan de trabajo y cronograma / Gantt

El Gantt tiene dos partes. La primera muestra el **programa del curso** (las fases del calendario del docente, sombreadas en azul). La segunda lista las **actividades del proyecto**: las ya terminadas llevan las fechas reales tomadas del historial del repositorio y se sombrean en verde; las demás se llenan por el equipo (columnas amarillas). Colores de estado: verde terminada, amarillo en proceso, gris pendiente. Las semanas comienzan en lunes.

Tabla 19. Programa del curso (Gantt programado)

Fase	Estado	Inicio	Fin	31/08	07/09	14/09	21/09	28/09	05/10	12/10	19/10	26/10	02/11	09/11	16/11	23/11	30/11
Integración de equipos e idea (11/09)	Programada																
Definición y diseño conceptual (25/09)	Programada																
Diseño técnico y planeación (09/10)	Programada																
Construcción e integración (23/10)	Programada																
Pruebas y validación (06/11)	Programada																
Entrega preliminar (20/11)	Programada																
Pre-integradora (26/11)	Programada																
Correcciones finales (27/11 al 02/12)	Programada																
Presentación final (03/12)	Programada																

Tabla 20. Actividades del proyecto (Gantt realizado)

Actividad	Estado	Inicio	Fin	31/08	07/09	14/09	21/09	28/09	05/10	12/10	19/10	26/10	02/11	09/11	16/11	23/11	30/11
Fase 0: arquitectura y decisiones (ADR)	**Terminada**	30/08	01/09														
Servidor, inventario y panel web	**Terminada**	01/09	02/09														
Visión: YOLO, ArUco y código	**Terminada**	01/09	02/09														
Robot Lumina (voz y cara)	**Terminada**	02/09	17/09														
Cambio de alcance: banda, fixtures y pistón (ADR 0010)	**Terminada**	16/09	16/09														

Actividad	Estado	Inicio	Fin	31/08	07/09	14/09	21/09	28/09	05/10	12/10	19/10	26/10	02/11	09/11	16/11	23/11	30/11
Sitio público y dominio	**Terminada**	16/10	16/10														
Puesta en marcha del Jetson Orin Nano Super	**Terminada**	21/10	21/10														
Agente de la celda, del Jetson y voz (R0 a R6)	**Terminada**	22/10	23/10														
Diseño 3D del carrito y hoja de armado rev. 3.8	**Terminada**																
LOGO! y pistón de la celda	**En proceso**																
Calibración de tiempos y prueba con la banda	**En proceso**																
Compra de materiales del carrito	**En proceso**																
Construcción de la estructura del carrito	**Pendiente**																
Montaje de la torre y del empujador	**Pendiente**																
Electrónica y cableado del carrito	**Pendiente**																
Firmware del ESP32 (PlatformIO)	**Pendiente**																
Enlace Modbus TCP Pi ↔ LOGO!	**Pendiente**																
Navegación por waypoints y acople	**Pendiente**																
Pruebas integrales celda → carrito	**Pendiente**																
Documento completo y presentación	**Pendiente**																

Fuente de las fechas verdes: historial de cambios del repositorio (fechas de los envíos de código). Una fecha de código no equivale a una prueba con hardware: la prueba física de cada actividad está en la sección 2.13.

3.2 Responsabilidades de los integrantes

Tabla 21. Responsabilidades por área

Área	Responsable	Qué entrega
Visión y software (Jetson, Pi, servidor)	responsable	Agentes, servidor, panel y modelo YOLO
Celda (LOGO!, banda, pistón)	responsable	Celda funcionando y su documentación
Diseño mecánico del carrito	responsable	Planos, estructura y torre
Electrónica y firmware del carrito	responsable	Tablero, cableado y firmware del ESP32
Interfaz y HMI	responsable	Pantallas y alarmas
Documentación e informe	Mario Iván Méndez Álvarez (elabora este documento)	Informe integrador

3.3 Recursos humanos, materiales, herramientas y servicios

Tabla 22. Recursos del proyecto

Tipo	Recurso	Estado
Humanos	Seis integrantes de 10.º cuatrimestre de Mecatrónica y el docente asesor	Disponibles
Materiales	Los de la lista de materiales (sección 2.10)	Ver disponibilidad en la lista
Herramientas	Impresión 3D (PETG) para escuadras y piezas; mesa de trabajo; nivel de burbuja; otras herramientas: multímetro, soldador, taladro, etc.	confirmar
Servicios de software	Repositorio privado de Git, Python, PlatformIO, Ollama (modelo local), Vosk (voz local)	Disponibles
Servicios en la nube	ElevenLabs (voz de Lumina) y GitHub Pages con Cloudflare (sitio público estático)	Disponibles; costo confirmar
Instalaciones	Laboratorio de la universidad y espacio de la celda confirmar	confirmar

3.4 Presupuesto y control de costos

La suma se hará con la lista de materiales de la sección 2.10. Resumen por subsistema (se llena cuando existan los costos):

Tabla 23. Presupuesto por subsistema y costo real

Subsistema	Presupuesto (MXN)	Costo real (MXN)
Estructura		
Tracción		

Subsistema	Presupuesto (MXN)	Costo real (MXN)
Energía y electrónica		
Torre y empujador		
Navegación y cómputo		
Celda de manufactura		
Tornillería y consumibles		
TOTAL		

3.5 Gestión de adquisiciones y disponibilidad de recursos

Tabla 24. Adquisiciones

Elemento	Estado de compra	Proveedor / cotización	Fecha prevista
Motores-rueda M0601 (2)	Modelo decidido y cotizado	proveedor	fecha
Perfil 3030 y tornillería	comprado / pendiente	proveedor	fecha
Electrónica (ESP32, drivers, sensores, módulos)	comprado / pendiente	proveedor	fecha
Batería y cargador	comprado / pendiente	proveedor	fecha
Materiales de la celda (relevadores, electroválvula, infrarrojo)	comprado / pendiente	proveedor	fecha

3.6 Riesgos, contingencias y acciones preventivas y correctivas

Tabla 25. Riesgos principales

Riesgo	Probabilidad	Efecto	Acción preventiva	Acción correctiva
Retraso en la llegada de materiales del carrito	Media	El carrito no se construye a tiempo	Priorizar la compra de los M0601 y la estructura	Entregar la celda funcionando y el carrito en simulación 3D
Falta del firmware del ESP32	Alta	El carrito no se mueve	Empezar por pruebas individuales de cada motor	Control manual por comandos desde el Jetson
Iluminación variable en la celda	Media	Juicios inestables	Cinco juicios iguales seguidos y luz controlada	Ajustar el umbral y la cámara
Falla del pistón sin aviso	Baja	Una pieza mala sigue en la línea	El código registra «fallo_actuacion» y levanta alarma	Revisar relevador y aire
Peso de la batería (8.3 kg) en un frenazo	Media	Daño al carrito	Correa con hebilla y cuna impresa	Reducir la velocidad máxima

3.7 Control y seguimiento: programado vs. realizado

Tabla 26. Control de avance contra el calendario

Fecha límite	Fase	Qué se esperaba	Qué se logró (24 de octubre de 2026)	Estado
11/09	Integración de equipos e idea	Idea con PLC, microcontrolador y manufactura flexible	Equipo integrado e idea definida	Cumplida
25/09	Definición y diseño conceptual	Arquitectura y diseño conceptual	Arquitectura y ADR; cambio de alcance el 16/09	Cumplida
09/10	Diseño técnico y planeación	Diseño técnico y lista de materiales	Diseño 3D del carrito (rev. 3.8) y lista de materiales sin costos	Cumplida en lo técnico; costos pendientes
23/10	Construcción e integración	Proyecto construido e integrado	Software de la celda y agentes integrados; carrito sin construir	Parcial
06/11	Pruebas y validación	Pruebas con el sistema armado	—	Pendiente
20/11	Entrega preliminar	Proyecto funcional y documento completo	—	Pendiente
26/11 y 03/12	Pre-integradora y presentación final	Presentaciones	—	Pendiente

Capítulo 4. Resultados y evaluación

Este capítulo reúne los resultados disponibles al 24 de octubre de 2026. Como el carrito aún no está construido, varias secciones tienen la estructura lista y los datos por completar tras las pruebas.

4.1 Resultados de las pruebas

Tabla 27. Resultados de las pruebas

Prueba	Resultado medido	Comentario
Pruebas automáticas del servidor y los agentes	285 pasan; 11 fallan por WinError 6 (subproceso)	VERIFICADO el 24/10/2026; test_main.py no se ejecutó (falta sqlalchemy en ese entorno)
Puesta en marcha del Jetson	PyTorch de NVIDIA funciona	Bitácora del repositorio, 21/10/2026
Pistón con el LOGO!	Funciona (según el equipo)	registro o video
Juicio con la banda real (20 + 20 piezas)		PENDIENTE
Tiempos del ciclo FAIL y PASS		PENDIENTE
Elevador, acople y navegación del carrito		PENDIENTE (carrito sin construir)

4.2 Pruebas en condiciones normales y, cuando aplique, máximas de operación

Tabla 28. Pruebas de operación

Condición	Qué se prueba	Resultado
Normal	Ciclo completo con una pieza PASS y una FAIL	
Normal	Transporte de un fixture de la banda a la mesa	
Máxima	Carga de 6 fixtures en la torre	
Máxima	Velocidad máxima del carrito (0.35 m/s) con frenado por obstáculo	
Límite	Batería baja: lectura del divisor de voltaje y comportamiento	
Falla	Pérdida de red o de la laptop: la celda sigue sola	

4.3 Cumplimiento de objetivos y metas

Tabla 29. Cumplimiento de objetivos

Objetivo específico	Estado	Evidencia
Juzgar piezas y accionar el pistón y la banda	Cumplido en software; prueba completa pendiente	Código y pruebas automáticas; pistón funciona

Objetivo específico	Estado	Evidencia
Diseñar y construir el carrito	Diseño cumplido; construcción pendiente	Figuras 5 a 8
Registrar y mostrar observaciones	Cumplido	Servidor y panel; capturas de la sección 4.7
Documentar diseño, materiales y pruebas	En proceso	Este documento

4.4 Impacto técnico, económico o productivo

Técnico: un modelo de integración de visión, PLC, microcontrolador y robot móvil documentado de punta a punta. **Económico:** todo el equipo es accesible para una pequeña empresa; el costo total se estima al llenar la lista de materiales: costo total. **Productivo:** una pieza se inspecciona en unos 3 segundos si pasa y unos 9 si se rechaza (tiempos de diseño, sin medir en la banda); el impacto productivo real se calcula con las pruebas de la sección 4.2: por medir.

4.5 Costo y tiempo real de realización

Tabla 30. Costo y tiempo: planeado y real

Concepto	Planeado	Real
Costo total del proyecto (MXN)		
Horas de trabajo del equipo		
Fecha de entrega del proyecto funcional	20/11/2026 (calendario del curso)	

4.6 Evaluación de factibilidad y cumplimiento normativo

Técnica: el software y la celda son factibles y ya corren; el carrito es factible según el diseño, pero depende de construirlo y escribir su firmware. **Económica:** pendiente de los costos. **Normativa:** los criterios de la sección 1.12 se aplicaron en el diseño; la evaluación formal de riesgos (ISO 12100) y la verificación del paro de emergencia son pendientes.

4.7 Producto final: fotografías, evidencia de funcionamiento y enlace al video

Las capturas del software (panel, celda, Lumina e inventario) están en la sección 2.12.3. Aquí va la evidencia física del producto final, que el equipo agrega cuando el sistema esté armado.

ESPACIO PARA FOTO O CAPTURA

Foto del producto final cuando esté armado.

Figura 25. Producto final funcionando: celda y carrito.

Enlace al video de funcionamiento: enlace al video — por completar

Conclusiones

El software de la línea y el diseño del carrito están avanzados y documentados con evidencia: la celda juzga y acciona, el servidor registra y el carrito tiene su diseño 3D, su lista de materiales y su diagrama eléctrico y de conexión. Lo más importante que falta es construir el carrito, escribir su firmware y probar el sistema completo con la banda real. Con los costos, fechas y fotos que el equipo debe agregar, el documento queda listo para la entrega preliminar.

Conclusión individual de cada integrante

Tabla 31. Conclusiones individuales (cinco renglones por integrante)

Integrante	Conclusión (5 renglones)
Mario Iván Méndez Álvarez	
Sadher Isaac Torres Galindo	
Pedro José Pallares Muñoz	
David Gerardo Garnica Chaparro	
Juan Emmanuel Martínez Aguirre	
Uriel Armando Rivero Rivero	

Referencias

Allegro MicroSystems. (s. f.). A4988: DMOS microstepping driver with translator and overcurrent protection [Hoja de datos].

Bosch Sensortec. (s. f.). BNO055: Intelligent 9-axis absolute orientation sensor [Hoja de datos].

Espressif Systems. (s. f.). ESP32 series datasheet [Hoja de datos].

Garrido-Jurado, S., Muñoz-Salinas, R., Madrid-Cuevas, F. J., & Marín-Jiménez, M. J. (2014). Automatic generation and detection of highly reliable fiducial markers under occlusion. Pattern Recognition, 47(6), 2280–2292.

International Organization for Standardization. (2010). Safety of machinery — General principles for design — Risk assessment and risk reduction (ISO 12100:2010).

International Organization for Standardization. (2023). Safety of machinery — Safety-related parts of control systems — Part 1 (ISO 13849-1).

Modbus Organization. (2012). Modbus application protocol specification V1.1b3.

NVIDIA Corporation. (2024). Jetson Orin Nano Super developer kit [Documentación técnica].

Raspberry Pi Ltd. (s. f.). Raspberry Pi 4 Model B datasheet [Hoja de datos].

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 779–788.

Secretaría del Trabajo y Previsión Social. (1999). NOM-004-STPS-1999, Sistemas de protección y dispositivos de seguridad en la maquinaria y equipo que se utilice en los centros de trabajo. Diario Oficial de la Federación.

Siemens AG. (s. f.). LOGO! manual [Manual de producto].

STMicroelectronics. (s. f.). VL53L0X: Time-of-flight ranging sensor [Hoja de datos].

Nota: los años «s. f.» deben completarse con la fecha de consulta y el enlace de cada documento antes de la entrega final.

Anexos

Anexo A. Códigos comentados

Fragmento comentado del programa de la celda (backend/cell_line.py): la secuencia que se ejecuta cuando hay cinco juicios iguales. Es el código real del repositorio, con comentarios que explican cada paso.

```
def _correr(self, juicio):
    accion = "pulso_no_enviado"
    try:
        if juicio == "FAIL":
            self._esperar(self.espera_piston)            # 3 s: la pieza llega al pistón
            if not self.salidas.poner(self.pin_piston, True):   # GPIO 8 en alto
                accion = "fallo_actuacion"               # no hay chip GPIO: no se finge
                return
            self._esperar(self.piston_encendido)         # 1 s con el pistón afuera
            self.salidas.poner(self.pin_piston, False)   # GPIO 8 en bajo
            self._esperar(self.piston_regreso)           # 2 s para que regrese
        self.salidas.poner(self.pin_banda, True)         # GPIO 9 -> entrada I4 del LOGO!
        self._esperar(self.pulso_banda)                  # pulso de 0.5 s
        self.salidas.poner(self.pin_banda, False)
        accion = "scrap_expulsado" if juicio == "FAIL" else "pieza_aceptada"
    finally:
        self.salidas.apagar()      # pase lo que pase, ambas salidas quedan apagadas
        self._anotar(juicio, accion)   # se registra la pieza y su acción
```

Anexo B. Planos y dibujos técnicos

Los planos disponibles son la vista lateral con cotas (Figura 8) y el modelo 3D con vistas armada, explosionada y transparente (Figuras 5 a 7). Los planos de fabricación acotados (cortes del perfil 3030 y piezas impresas) se exportan de la hoja de armado: agregar planos exportados.

Anexo C. Diagramas adicionales

Se incluyen en el cuerpo del documento los diagramas de flujo (Figuras 1 a 3), de arquitectura (Figura 4), eléctricos (Figuras 9 y 10), de conexiones del ESP32 (Figuras 13 y 14) y de tiempos (Figura 12). Falta agregar el diagrama del programa del LOGO!: agregar diagrama.

Anexo D. Fichas técnicas y manuales relevantes

Tabla 32. Fichas técnicas relevantes

Componente	Documento	Estado
ESP32 DevKitC	Hoja de datos de Espressif	agregar copia
A4988	Hoja de datos de Allegro	agregar copia
VL53L0X	Hoja de datos de STMicroelectronics	agregar copia
BNO055	Hoja de datos de Bosch Sensortec	agregar copia
LOGO!	Manual de Siemens	agregar copia
Jetson Orin Nano Super	Documentación de NVIDIA	agregar copia
Raspberry Pi 4	Hoja de datos de Raspberry Pi Ltd	agregar copia

Componente	Documento	Estado
Motor-rueda M0601	Ficha del proveedor	agregar copia

Anexo E. Formatos de pruebas y listas de verificación

Formato para registrar cada prueba de la sección 2.13. Se imprime una hoja por prueba.

Tabla 33. Formato de registro de pruebas

Campo	Registro
Prueba (número y nombre)	
Fecha y hora	
Responsable	
Equipo y versión de código	
Condiciones (iluminación, piezas, voltaje)	
Resultado medido	
Criterio de aceptación	
Aprobada (sí / no)	
Observaciones y foto	

Tabla 34. Lista de verificación antes de energizar el carrito

N.º	Revisión	Hecho
1	Polaridad de la batería y de cada fusible revisada	
2	Paro de emergencia suelto y relevador cerrado	
3	Tensión del reductor de 5 V entre 5.0 y 5.1 V	
4	Ruedas en el aire para la primera prueba de movimiento	
5	Cables sin tensión y sin cruzar con los motores	
6	Correa de la batería apretada	

Anexo F. Códigos de programación

El código completo está en el repositorio privado del equipo. La tabla indica dónde está cada parte; el firmware del ESP32 todavía no existe.

Tabla 35. Mapa del código

Ruta	Contenido
backend/cell_line.py	Secuencia de la celda: juicio estable, pistón y banda por GPIO

Ruta	Contenido
backend/celda_agent.py y backend/cell_hmi.py	Agente de la celda para la Raspberry Pi 4 y datos de la HMI
backend/aether_agent.py, agent_*.py y observation_*.py	Agente del Jetson, contratos y envío de observaciones en lotes
backend/main.py, backend/db/	Servidor FastAPI y base de datos SQLite
backend/voice/	Servicio de voz (micrófono local y Vosk)
dashboard/	Panel web: portada, celda, inventario, Lumina y dataset
tests/	Pruebas automáticas (pytest)
firmware/	Reservada para el ESP32: PENDIENTE (vacía)

Los fragmentos explicados están en la sección 2.9 y en el Anexo A.